# FACULTY OF INFORMATION TECHNOLOGY

## 2026 Mobile App Development 700

## ASSIGNMENT

RICHFIELD

**Name and Surname:** Dihan Visagie **Assignment due date:** 23 October 2026 **Date submitted:** 23 October 2026

**Students ITS No:** 402111461

**Qualification:** BSc IT **Year of Study:** Third Year **Semester:** First Semester

| | | | |
|---|---|---|---|
| ONE (40 MARKS) | | | |
| TWO (30 MARKS) | | | |
| Three (30 MARKS) | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

## ASSIGNMENT INSTRUCTIONS

***Please tick each box to confirm completion. (10 Marks)***

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

**For essay-style assignments, please include the following sections:**

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

**DECLARATION OF ORIGINALITY:**

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

**Student Signature:** Dihan Visagie **Date:** 20 April 2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

# Contents


Background and problem:.....3

Purpose of the application:.....3

Target users:.....3

Scope and key features:.....3

Development tools and approach:.....4

1. SYSTEM DESIGN:.....4

Application architecture:.....4

Screen flow:.....5

Data model:.....6

Relationships and design decisions.....6

Strict-matching algorithm:.....7

DATABASE CHOICE SQLITE:.....7

2. SCREENSHOTS OF RUNNING APP :.....8

3. KEY CODE SNIPPETS:.....34

Strict-matching rule RecipeMatcher.java:.....34

Database creation and seeding DatabaseHelper.java:.....35

CRUD with parameterised queries PantryDao.java:.....35

Intents and the Activity lifecycle PantryListActivity.java:.....36

Input validation AddEditIngredientActivity.java:.....37

4. CHALLENGES AND THE SOLUTION:.....38

Template code blocked the build and unit tests:.....38

App crashed on launch because of an SQL syntax error:.....39

Making strict matching robust to real-world input:.....39

5. CONCLUSION:.....39

Conclusion:.....39

What I learned:.....40

What I would improve with more time:.....40

## INTRODUCTION:

### Background and problem:

Food waste is a serious problem in South Africa. A study by the Council for Scientific and Industrial Research found that about 45% of the country's available food supply is lost or wasted, roughly 10 million tonnes every year (CSIR, 2021). Globally, households produce the largest share of food waste, around 60% of the total (UNEP, 2024). This means everyday decisions at home make a real difference.

A common reason food is thrown away at home is simple: people forget what they already have, or they cannot think of a meal that uses the leftovers in the cupboard. Instead, they buy more food, and the leftover ingredients expire. Most recipe apps make this worse. They show recipes that need several extra ingredients, which sends the user back to the shop.

### Purpose of the application:

Smart Pantry Manager is a Java Android application that helps users reduce food waste. The user records the ingredients they have at home (their pantry), and the app suggests recipes they can cook using only those ingredients. A recipe is only suggested when the user genuinely has every ingredient it needs, in at least the required quantity. No shopping trip is required. This strict-matching rule is the core business logic of the application.

### Target users:

- Students and young professionals living alone or in shared accommodation, who often have small amounts of leftover ingredients and a limited budget.
- Families and budget-conscious households that want to use up what they have before buying more.
- Anyone who wants to waste less food, especially ingredients that are close to their expiry date.

### Scope and key features:

The application provides the following features, as required by the assignment brief:

- Pantry management: add, view, edit and delete ingredients (name, quantity, unit and an optional expiry date), with full CRUD stored in SQLite.
- Recipe collection: 20 recipes, including South African dishes such as Chakalaka and Pap, loaded into the database automatically on first launch.
- Suggested Recipes: applies the strict-matching rule and lists only the recipes the user can cook right now.
- Recipe detail: shows the full ingredient list and a numbered method.
- Settings: expiring-soon alerts, the warning period, a default unit and a clear-pantry option.

- Feedback and validation: clear empty-state messages instead of blank screens, and validation errors on the data-entry form.

In line with the brief, the application does not use Google Maps, any mapping SDK or GPS/location services, does not process payments, and is not published to the Google Play Store. The app requests no Android permissions at all.

Development tools and approach:

The application was built entirely in Java using Android Studio, with XML layouts and Material Design 3 components. Data is stored on-device using SQLite through Android's SQLiteOpenHelper. The project was developed under Git version control and pushed to GitHub with incremental commits: https://github.com/Scorpionstriker/SPMMadAssigment . The strict-matching logic is covered by JUnit 4 unit tests.

# 1. SYSTEM DESIGN:

Application architecture:

The code is organised into packages by responsibility. Activities handle only the user interface and navigation. Database access, business logic and data classes are kept in separate packages. This makes the code easier to read and maintain, and it allows the matching logic to be unit-tested without an emulator, because it contains no Android code.

| Package | Responsibility | Main classes |
|---|---|---|
| activities | Screens, user input and navigation with Intents | PantryListActivity, AddEditIngredientActivity, SuggestedRecipesActivity, RecipeDetailActivity, SettingsActivity |
| adapters | Bind lists of data to RecyclerViews | PantryAdapter, RecipeAdapter, IngredientAdapter |
| database | All SQLite code: schema, creation, CRUD and seed data | DatabaseSchema, DatabaseHelper, PantryDao, RecipeDao, RecipePreLouder |
| logic | Business logic in plain Java (no Android code) | RecipeMatcher, IngredientManager, UnitManager |

| | | |
|---|---|---|
| models | Data classes representing table rows | PantryItem, Recipe, RecipeIngredient |
| utils | Shared helpers | NavigationHelper, PreferencesManager, DateUtils, FormatUtils |

Screen flow:

The application has five screens, each implemented as a separate Activity. The three main screens (Pantry, Recipes and Settings) are connected by a bottom navigation bar. Two detail screens are opened with explicit Intents that pass the database ID of the selected record as an extra. Figure 1 shows how the screens connect.

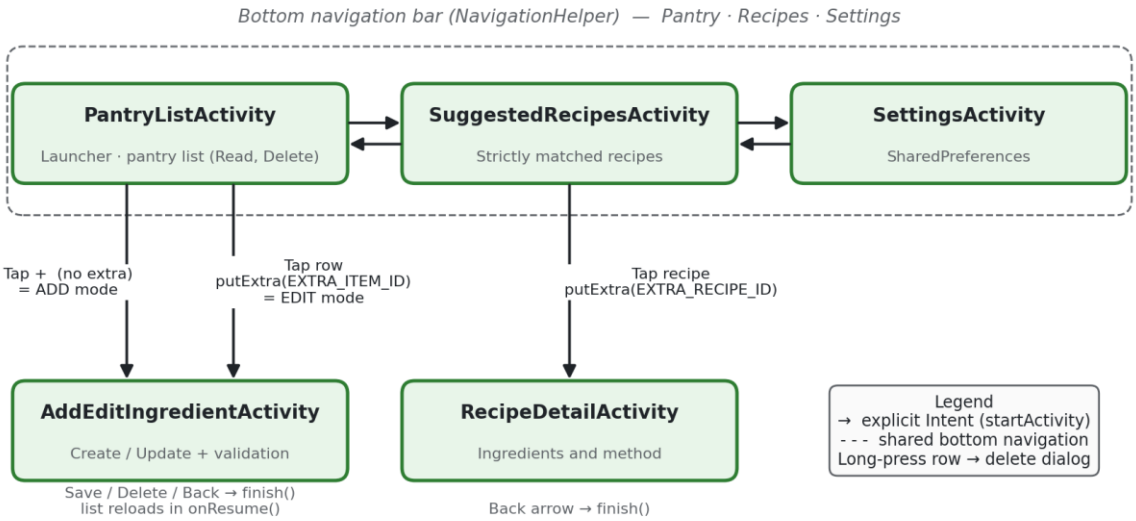



*Figure 1: Screen flow diagram showing the five Activities and the Intents between them*

- PantryListActivity is the launcher screen. Tapping + opens AddEditIngredientActivity with no extra (add mode). Tapping a row passes EXTRA_ITEM_ID (edit mode). Long-pressing a row shows a delete confirmation dialog.
- SuggestedRecipesActivity runs the matching logic. Tapping a recipe opens RecipeDetailActivity, passing EXTRA_RECIPE_ID.
- Lifecycle: both list screens reload their data in onResume(). When the user returns from the add/edit screen or switches tabs, the lists are therefore always up to date.

Data model:

The database contains three tables. Figure 2 shows the entity-relationship diagram.

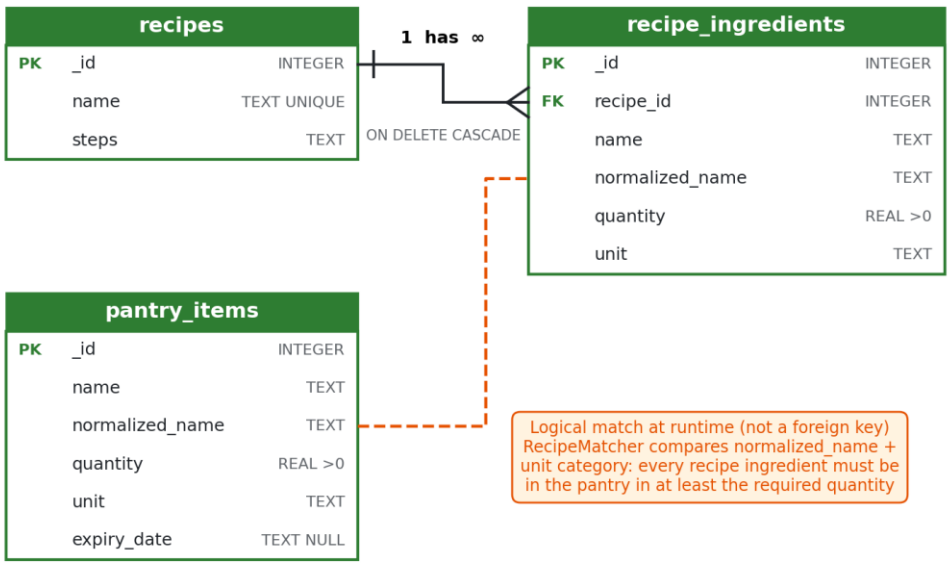



*Figure 2: Entity-relationship diagram of the SQLite database*

| Table | Purpose |
|---|---|
| pantry_items | Ingredients the user currently has at home |
| recipes | Recipe name and preparation steps |
| recipe_ingredients | Each ingredient a recipe needs, with quantity and unit |

Relationships and design decisions

- One-to-many: one recipe has many ingredient rows. This is enforced with a foreign key (recipe_id) and ON DELETE CASCADE. SQLite has foreign keys switched off by default, so they are enabled in DatabaseHelper.onConfigure().
- Logical match, not a foreign key: pantry_items and recipe_ingredients are compared at runtime on normalized_name and unit category. A foreign key would be wrong here, because a recipe must be able to require an ingredient the user does not have, and deleting a pantry item must never affect a recipe.
- Normalised names are stored: each name is converted to a matching key when it is saved (for example "Tomatoes" becomes "tomato"). The original name is kept for display.

- Constraints: CHECK (quantity > 0), NOT NULL on required columns, and UNIQUE recipe names protect data integrity at database level.
- Settings are stored in SharedPreferences, not in a table, because they are simple key-value pairs.

Strict-matching algorithm:

The strict-matching rule is implemented in RecipeMatcher and works in four steps:

1. Normalise names (IngredientManager): lowercase, trim, make the last word singular and apply synonyms, e.g. "TOMATOES" → "tomato" and "Mielie meal" → "maize meal".
2. Convert units (UnitManager): every amount is converted to a base unit for its category: grams (mass), millilitres (volume) or pieces (count). For example, 1 kg becomes 1000 g. Amounts in different categories, such as grams and cups, are never compared.
3. Build a stock table from the pantry: name|category → total amount. Duplicate pantry rows are added together.
4. Check every ingredient: a recipe qualifies only if every required ingredient is found and the available amount is at least the required amount. One missing or insufficient ingredient excludes the recipe.

| Pantry contents | Cheese Omelette needs | Result |
|---|---|---|
| Eggs 6 pcs, Cheddar 200 g, Butter 250 g, Milk 1 L | Egg 3 pcs, Cheese 50 g, Butter 10 g, Milk 30 ml | Suggested: all 4 ingredients are covered (1 L = 1000 ml ≥ 30 ml; "Cheddar" is a synonym for cheese) |
| Eggs 6 pcs, Cheddar 200 g, Butter 250 g | Egg 3 pcs, Cheese 50 g, Butter 10 g, Milk 30 ml | Not suggested: milk is missing (3 of 4 ingredients) |
| Eggs 2 pcs, Cheddar 200 g, Butter 250 g, Milk 1 L | Egg 3 pcs, Cheese 50 g, Butter 10 g, Milk 30 ml | Not suggested: not enough eggs (2 < 3) |

DATABASE CHOICE SQLITE:

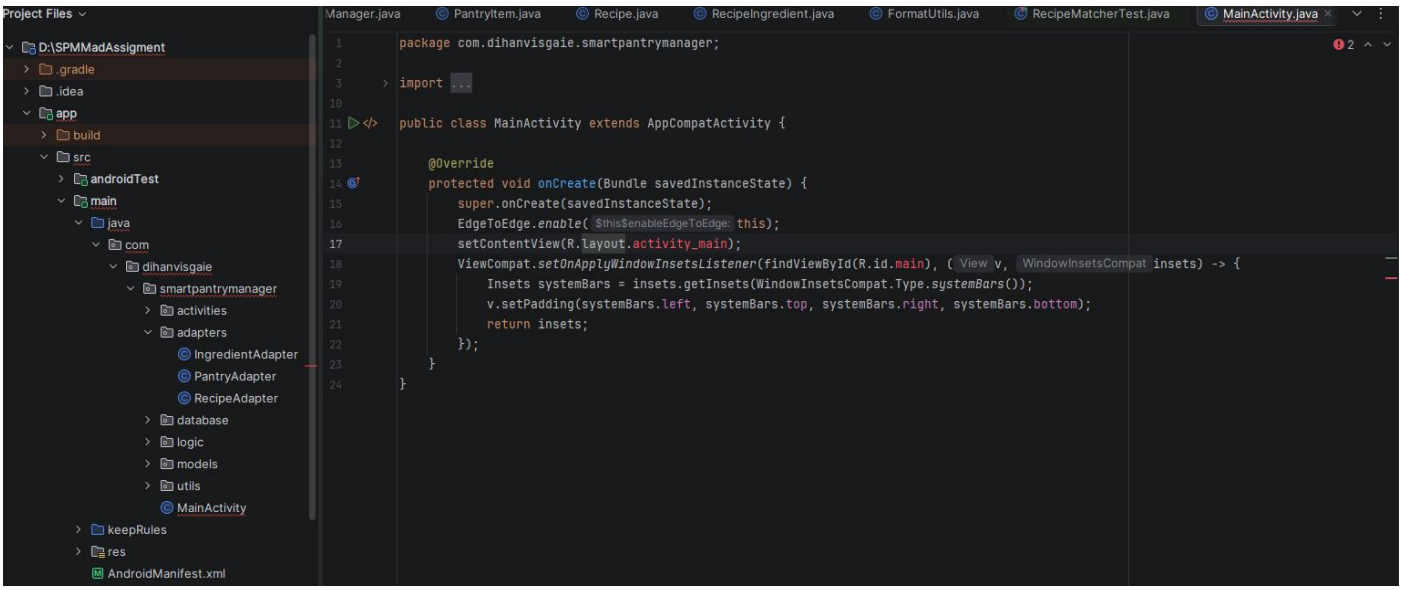
SQLite was chosen over Firebase and PostgreSQL for four reasons:

- The data is personal and local. A pantry belongs to one person on one device, so there is nothing to synchronise between users or devices.
- It works offline, with no internet connection, account, server or hosting costs.
- It is built into Android, so no extra libraries, backend or configuration are needed. It is also the approach covered in the module's persistent-data chapter.

- It genuinely persists. Data is stored in a database file in the app's private storage, so it survives closing the app and restarting the device.

## 2. SCREENSHOTS OF RUNNING APP :

The following screenshots were taken from the running application on an Android emulator.






```
26-10-22 14:42:33.237  8225-8225  AndroidRuntime          com.dihanvisgaie.smartpantrymanager  E  FATAL EXCEPTION: main
                                                                                                Process: com.dihanvisgaie.smartpantrymanager, PID: 8225
                                                                                                java.lang.RuntimeException: Unable to resume activity {com.dihanvisgaie.smartpantrymanager/com.dihanvisgaie.smartpantryma
                                                                                                    at android.app.ActivityThread.performResumeActivity(ActivityThread.java:5973)
                                                                                                    at android.app.ActivityThread.handleResumeActivity(ActivityThread.java:6006)
                                                                                                    at android.app.servertransaction.ResumeActivityItem.execute(ResumeActivityItem.java:73)
                                                                                                    at android.app.servertransaction.ActivityTransactionItem.execute(ActivityTransactionItem.java:63)
                                                                                                    at android.app.servertransaction.TransactionExecutor.executeLifecycleItem(TransactionExecutor.java:169)
                                                                                                    at android.app.servertransaction.TransactionExecutor.executeTransactionItems(TransactionExecutor.java:101)
                                                                                                    at android.app.servertransaction.TransactionExecutor.execute(TransactionExecutor.java:80)
                                                                                                    at android.app.ActivityThread$H.handleMessage(ActivityThread.java:3032)
                                                                                                    at android.os.Handler.dispatchMessageImpl(Handler.java:142)
                                                                                                    at android.os.Handler.dispatchMessage(Handler.java:125)
                                                                                                    at android.os.Looper.loopOnce(Looper.java:296)
                                                                                                    at android.os.Looper.loop(Looper.java:397)
                                                                                                    at android.app.ActivityThread.main(ActivityThread.java:9523) <1 internal line>
                                                                                                    at com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:575)
                                                                                                    at com.android.internal.os.ZygoteInit.main(ZygoteInit.java:939)
                                                                                                Caused by: android.database.sqlite.SQLiteException: near "CASE": syntax error (code 1 SQLITE_ERROR): , while compiling: S
                                                                                                    at android.database.sqlite.SQLiteConnection.nativePrepareStatement(Native Method)
                                                                                                    at android.database.sqlite.SQLiteConnection.-$$Nest$smnativePrepareStatement(SQLiteConnection.java:0)
                                                                                                    at android.database.sqlite.SQLiteConnection$PreparedStatementCache.createStatement(SQLiteConnection.java:1627)
                                                                                                    at android.database.sqlite.SQLiteConnection.acquirePreparedStatementLI(SQLiteConnection.java:1117)
                                                                                                    at android.database.sqlite.SQLiteConnection.acquirePreparedStatement(SQLiteConnection.java:1145)
                                                                                                    at android.database.sqlite.SQLiteConnection.prepare(SQLiteConnection.java:708)
                                                                                                    at android.database.sqlite.SQLiteSession.prepare(SQLiteSession.java:625)
                                                                                                    at android.database.sqlite.SQLiteProgram.<init>(SQLiteProgram.java:62)
                                                                                                    at android.database.sqlite.SQLiteQuery.<init>(SQLiteQuery.java:37)
```

```
BUILD SUCCESSFUL in 8s
22 actionable tasks: 5 executed, 17 up-to-date
Configuration cache entry reused.

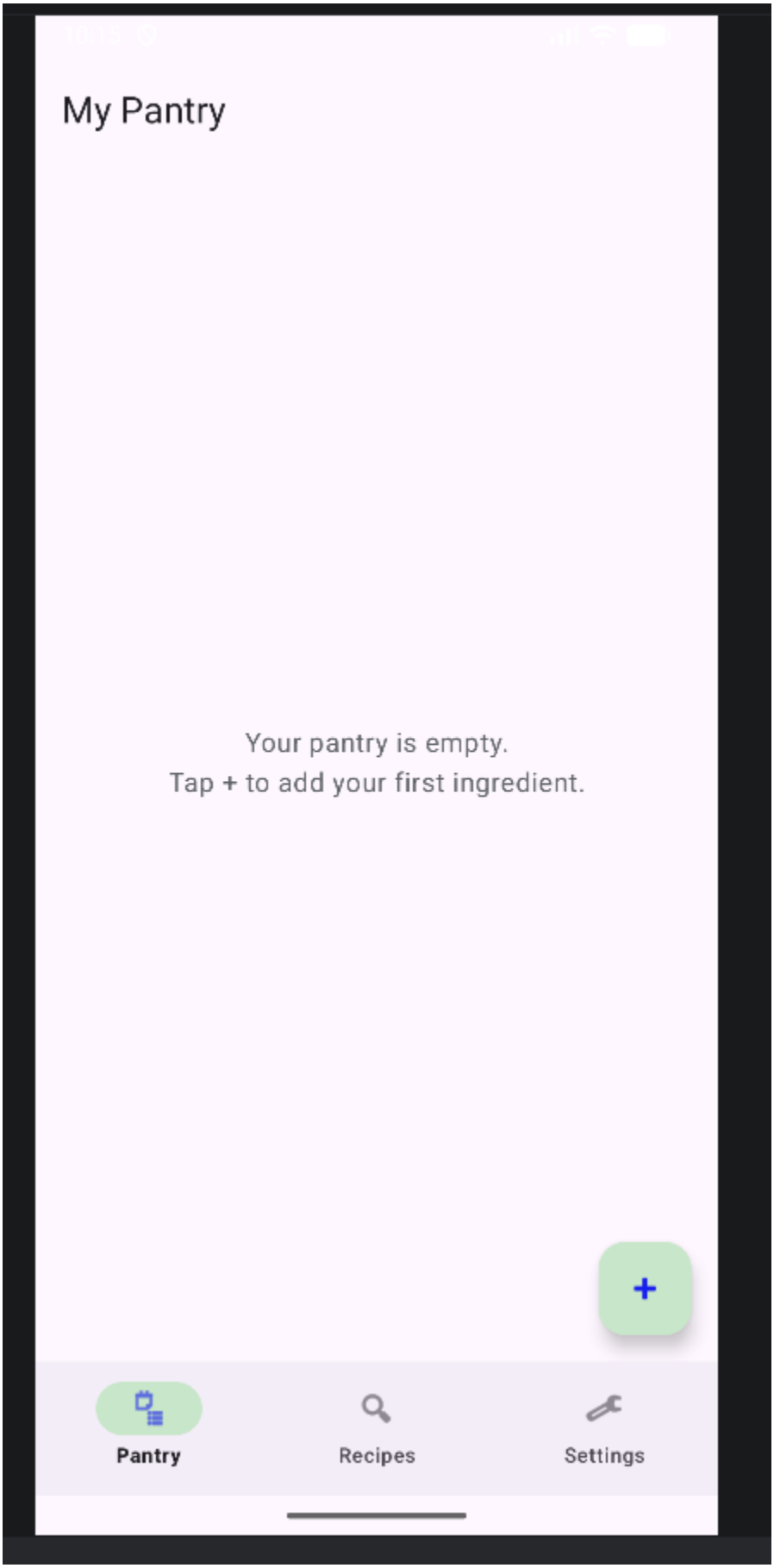
Build Analyzer results available
08:33:13: Execution finished ':app:testDebugUnitTest --tests "com.dihanvisgaie.smartpantrymanager.logic.RecipeMatcherTest"'.
```

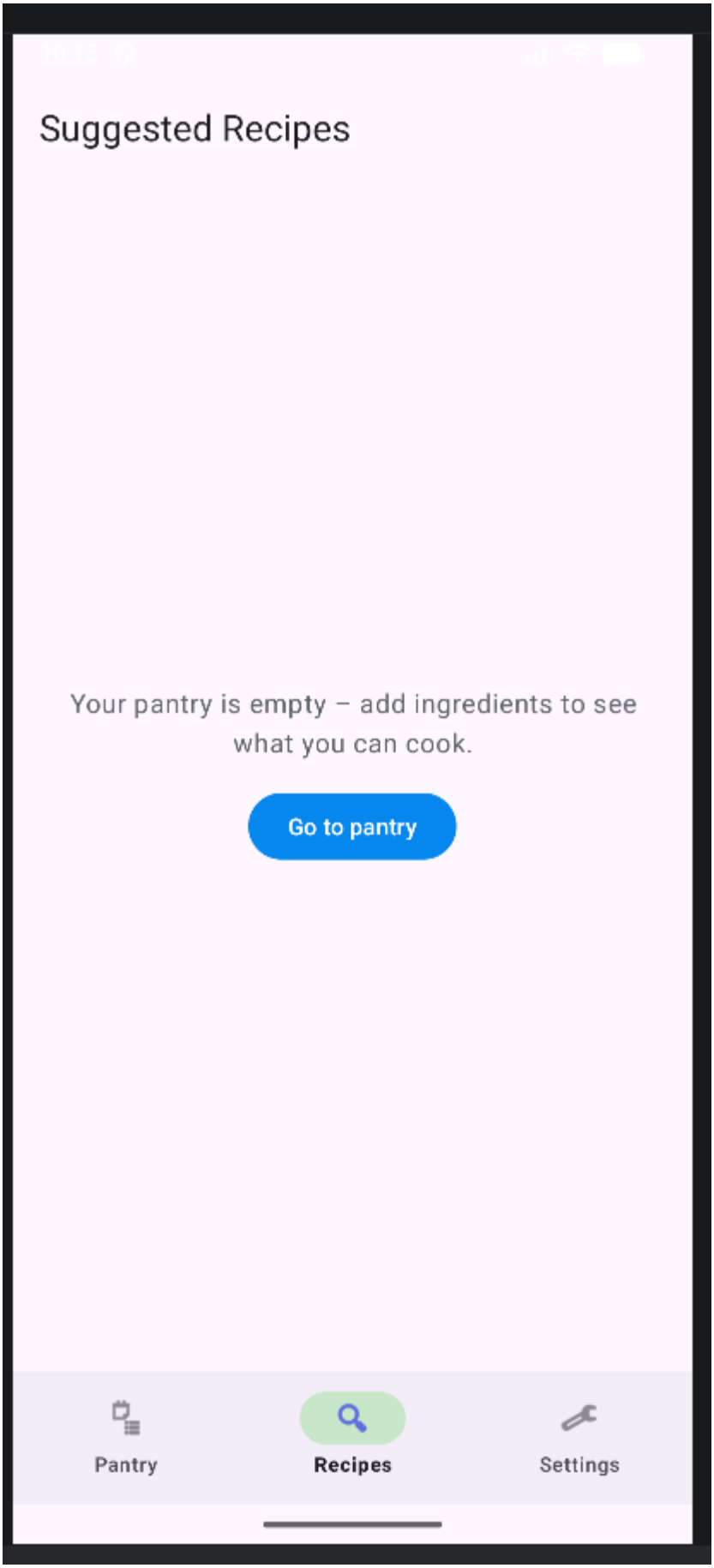
```
Run  RecipeMatcherTest

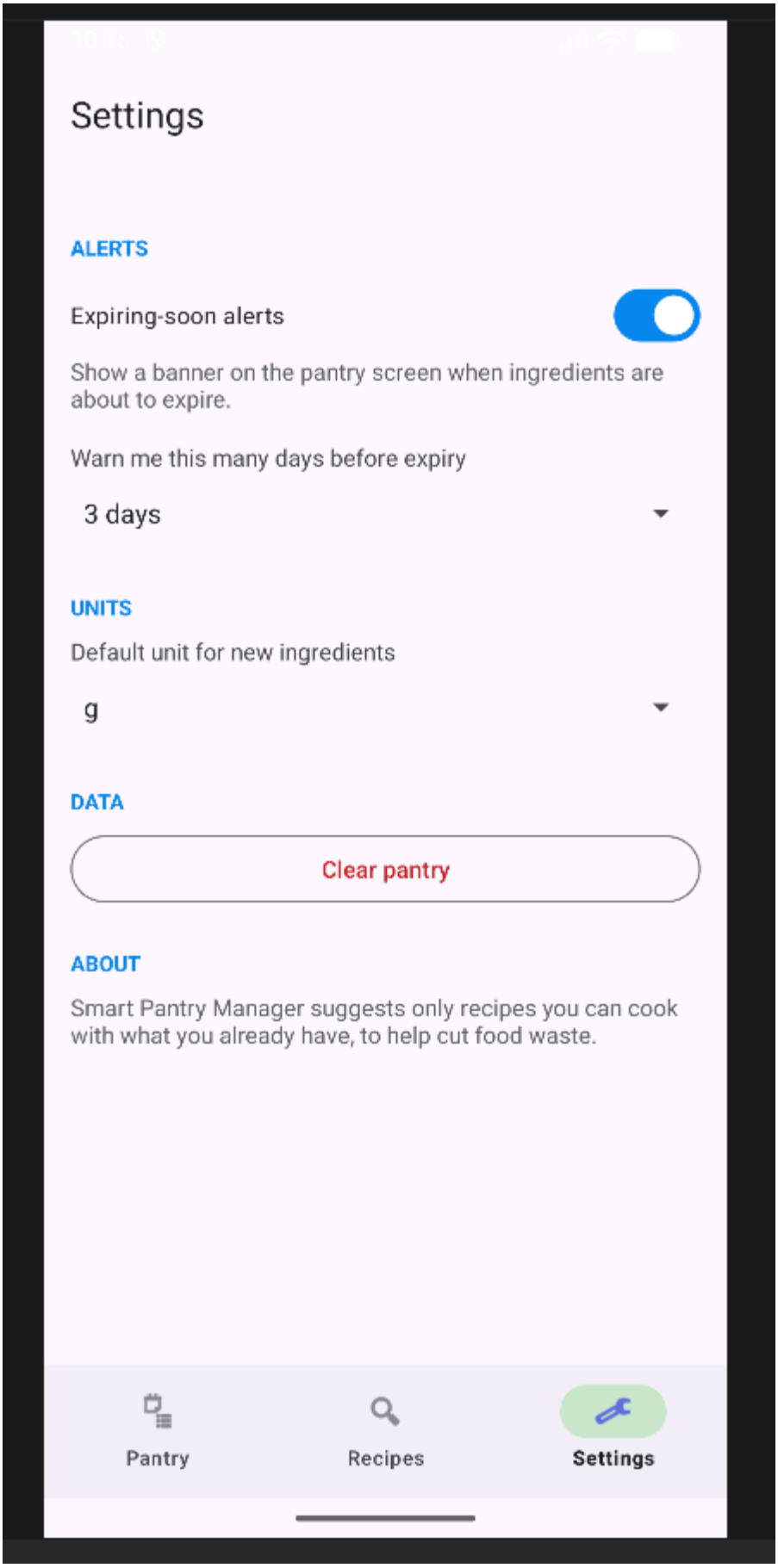
Test Results  8 sec 646 ms
11 tests passed  11 tests total, 8 sec 646 ms

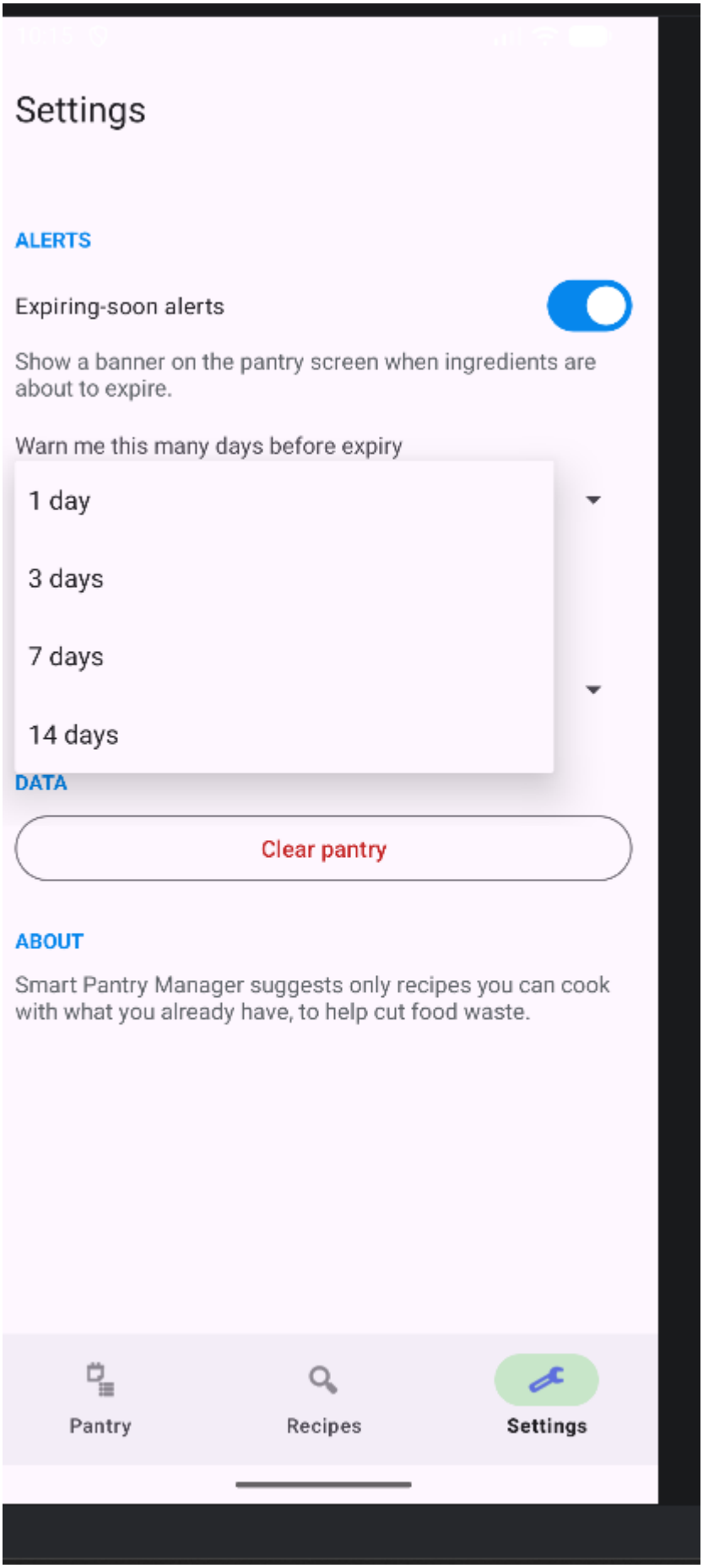
Executing tasks: [:app:testDebugUnitTest, --tests, com.dihanvisgaie.smartpantrymanager.logic.RecipeMatcherTest] in project D:\SPMMadAssignment

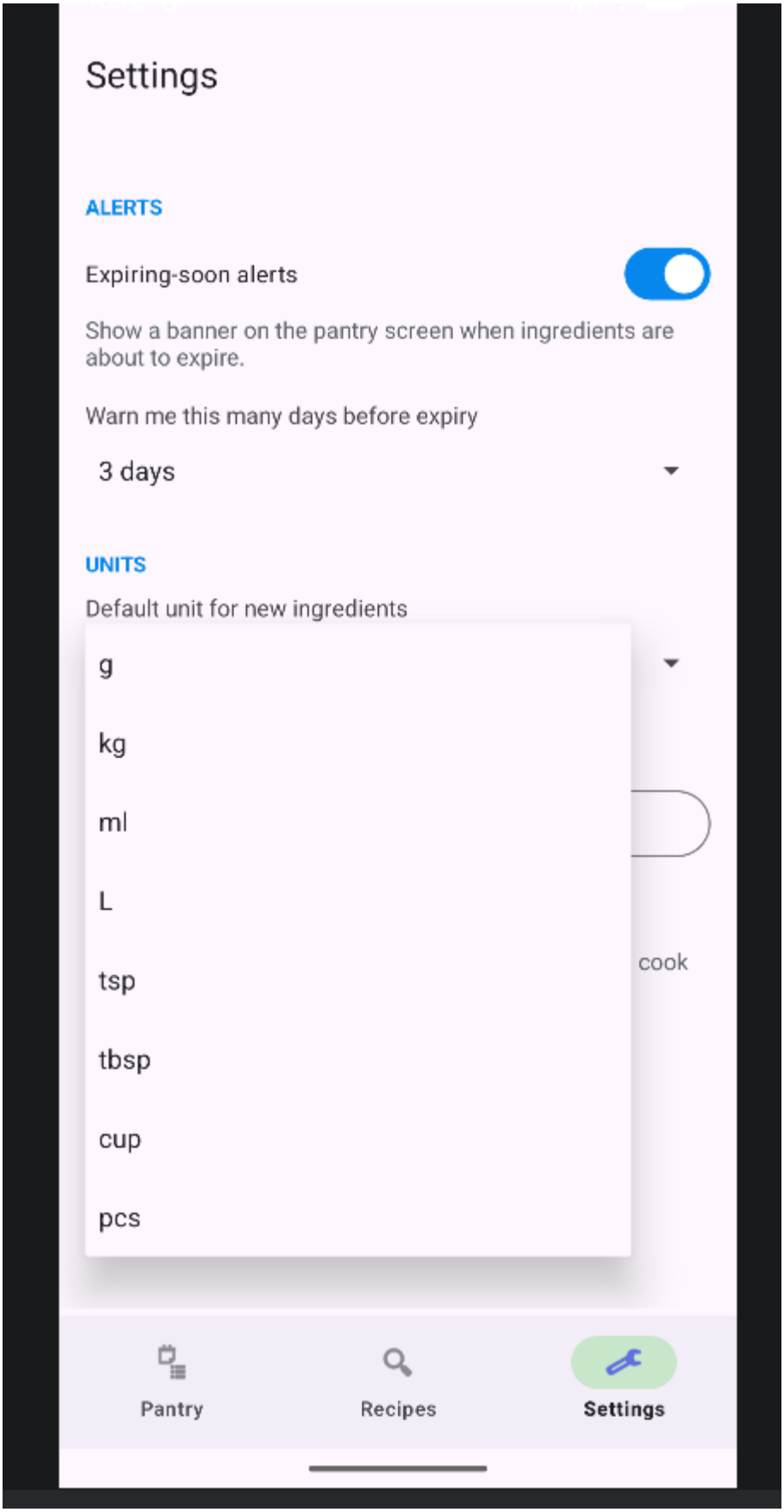
Reusing configuration cache.
> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:preDebugUnitTestBuild UP-TO-DATE
> Task :app:createDebugCompatibleScreenManifests UP-TO-DATE
> Task :app:generateDebugResources UP-TO-DATE
> Task :app:checkDebugAarMetadata UP-TO-DATE
> Task :app:processDebugNavigationResources UP-TO-DATE
> Task :app:javaPreCompileDebug UP-TO-DATE
> Task :app:javaPreCompileDebugUnitTest UP-TO-DATE
> Task :app:extractDeepLinksDebug UP-TO-DATE
> Task :app:mapDebugSourceSetPaths UP-TO-DATE
> Task :app:compileDebugNavigationResources UP-TO-DATE
> Task :app:mergeDebugResources UP-TO-DATE
> Task :app:packageDebugResources UP-TO-DATE
> Task :app:parseDebugLocalResources UP-TO-DATE
> Task :app:generateDebugRFile UP-TO-DATE
> Task :app:processDebugMainManifest UP-TO-DATE
> Task :app:processDebugManifest UP-TO-DATE
> Task :app:processDebugManifestForPackage UP-TO-DATE
> Task :app:processDebugResources UP-TO-DATE
> Task :app:compileDebugKotlin NO-SOURCE
> Task :app:processDebugJavaRes NO-SOURCE
> Task :app:compileDebugJavaWithJavac
> Task :app:bundleDebugClassesToCompileJar
> Task :app:bundleDebugClassesToRuntimeJar
> Task :app:compileDebugUnitTestKotlin NO-SOURCE
> Task :app:processDebugUnitTestJavaRes NO-SOURCE
> Task :app:compileDebugUnitTestJavaWithJavac
> Task :app:testDebugUnitTest
```

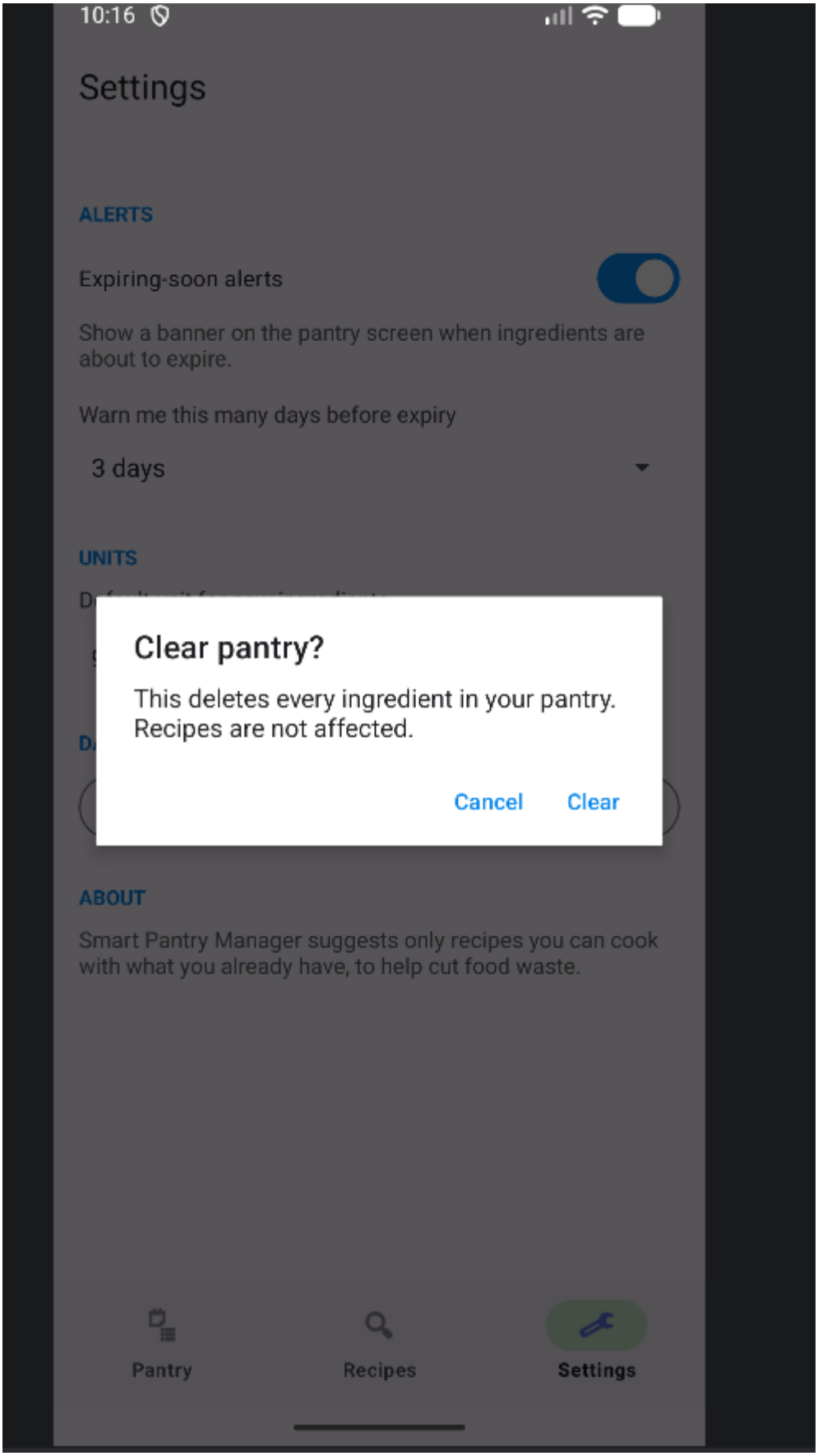

My Pantry
Your pantry is empty.
Tap + to add your first ingredient.
+
Pantry
Recipes
Settings

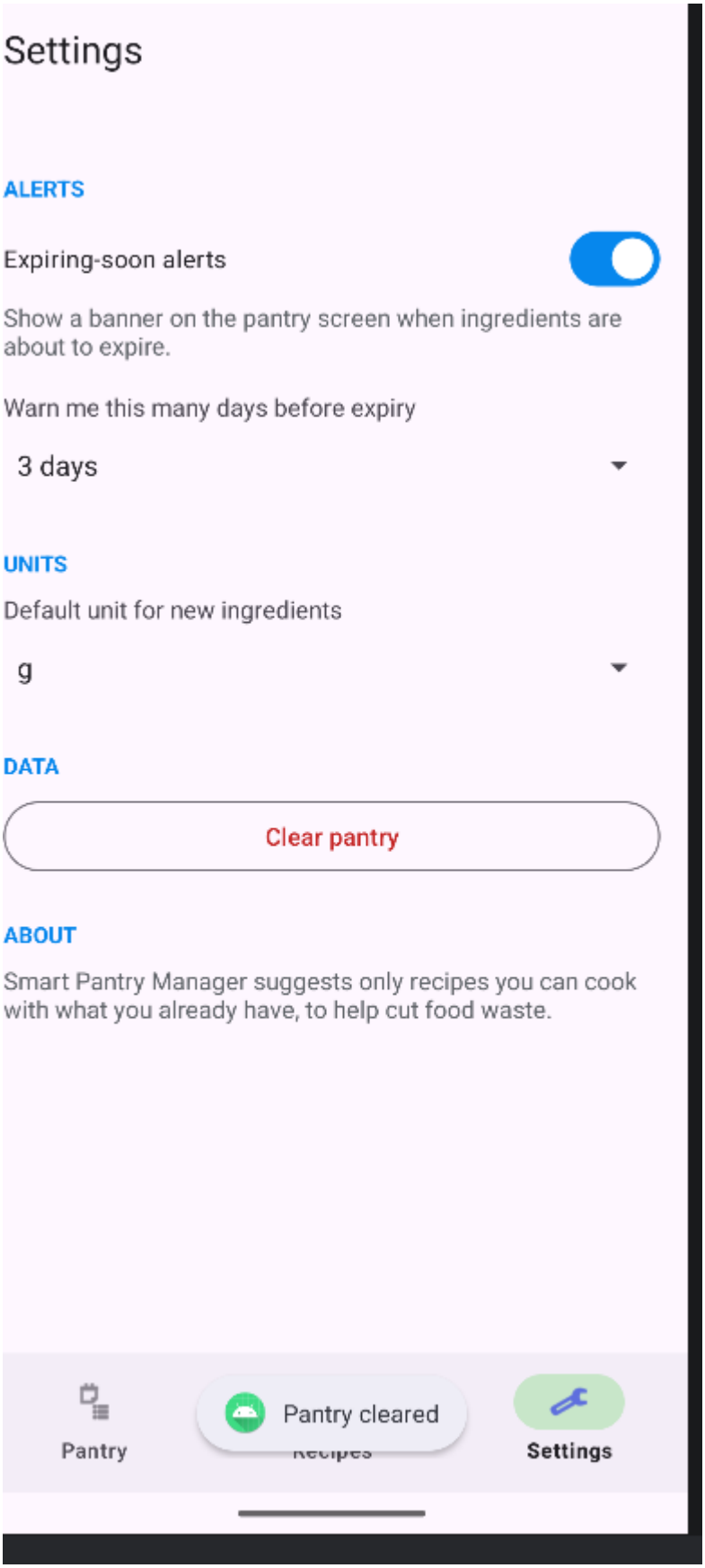

Suggested Recipes
Your pantry is empty – add ingredients to see what you can cook.
Go to pantry
Pantry
Recipes
Settings

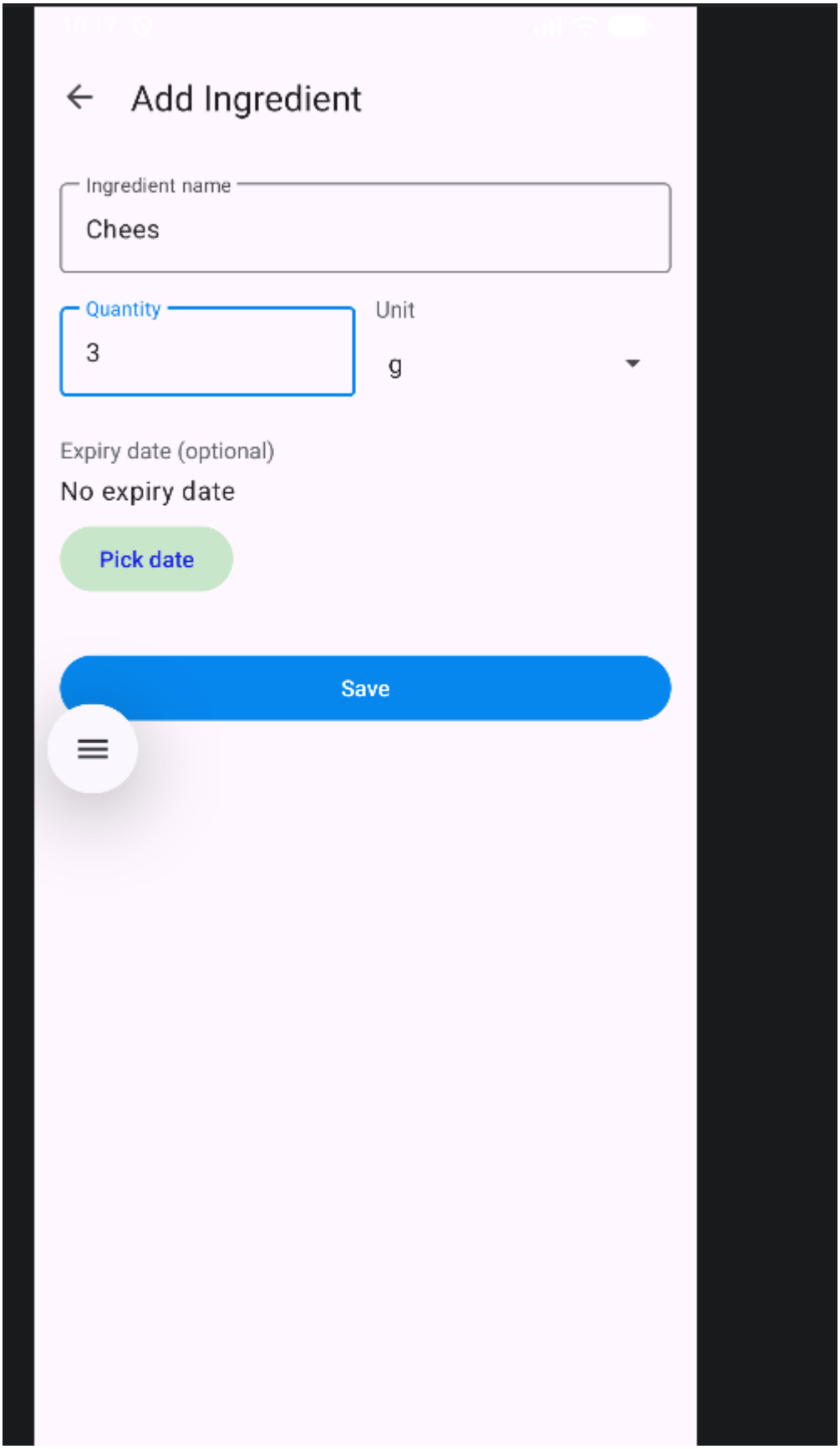

Settings
ALERTS
Expiring-soon alerts
Show a banner on the pantry screen when ingredients are about to expire.
Warn me this many days before expiry
3 days
UNITS
Default unit for new ingredients
g
DATA
Clear pantry
ABOUT
Smart Pantry Manager suggests only recipes you can cook with what you already have, to help cut food waste.
Pantry
Recipes
Settings

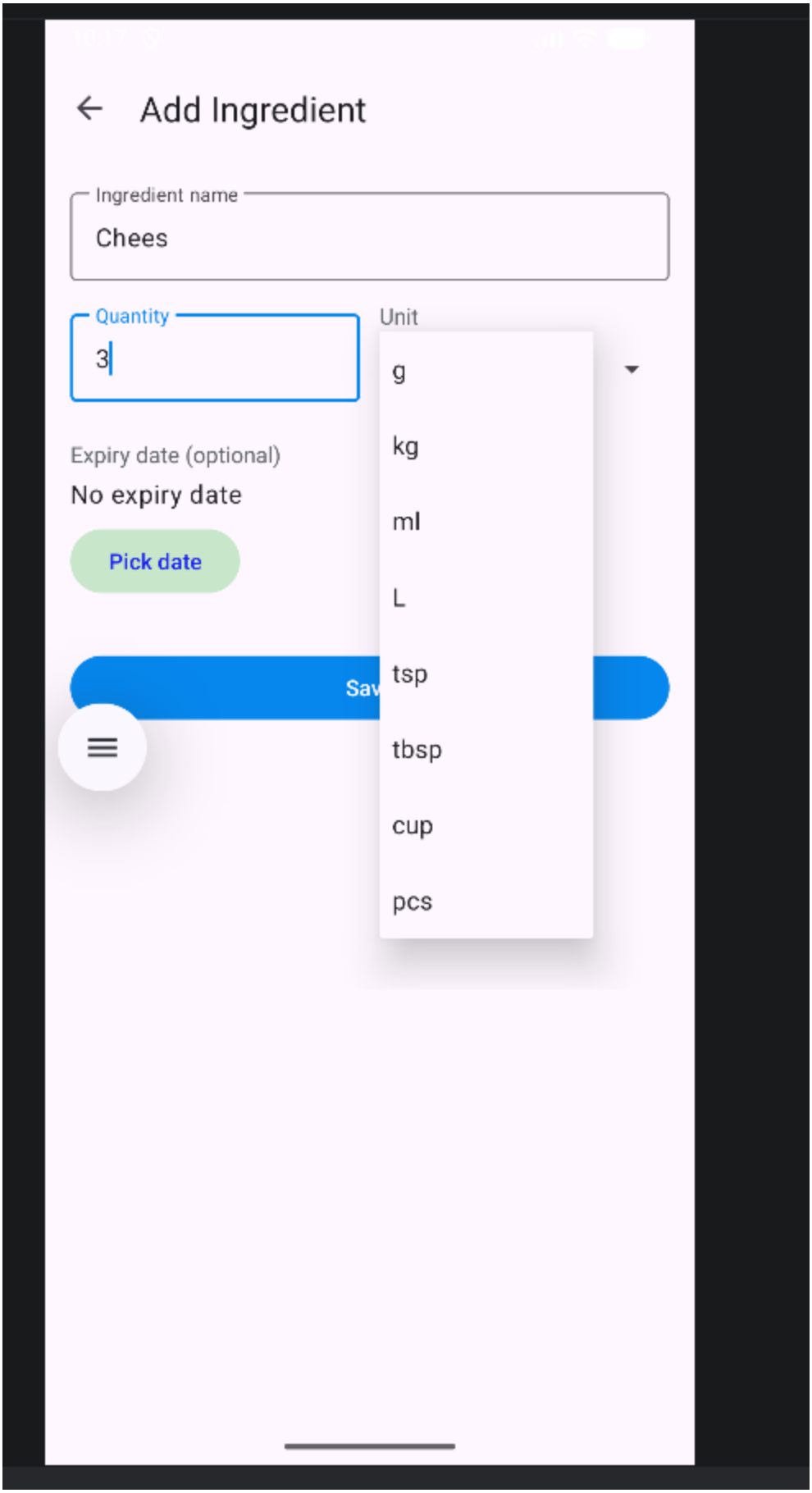

Settings
ALERTS
Expiring-soon alerts
Show a banner on the pantry screen when ingredients are about to expire.
Warn me this many days before expiry
1 day
3 days
7 days
14 days
DATA
Clear pantry
ABOUT
Smart Pantry Manager suggests only recipes you can cook with what you already have, to help cut food waste.
Pantry
Recipes
Settings

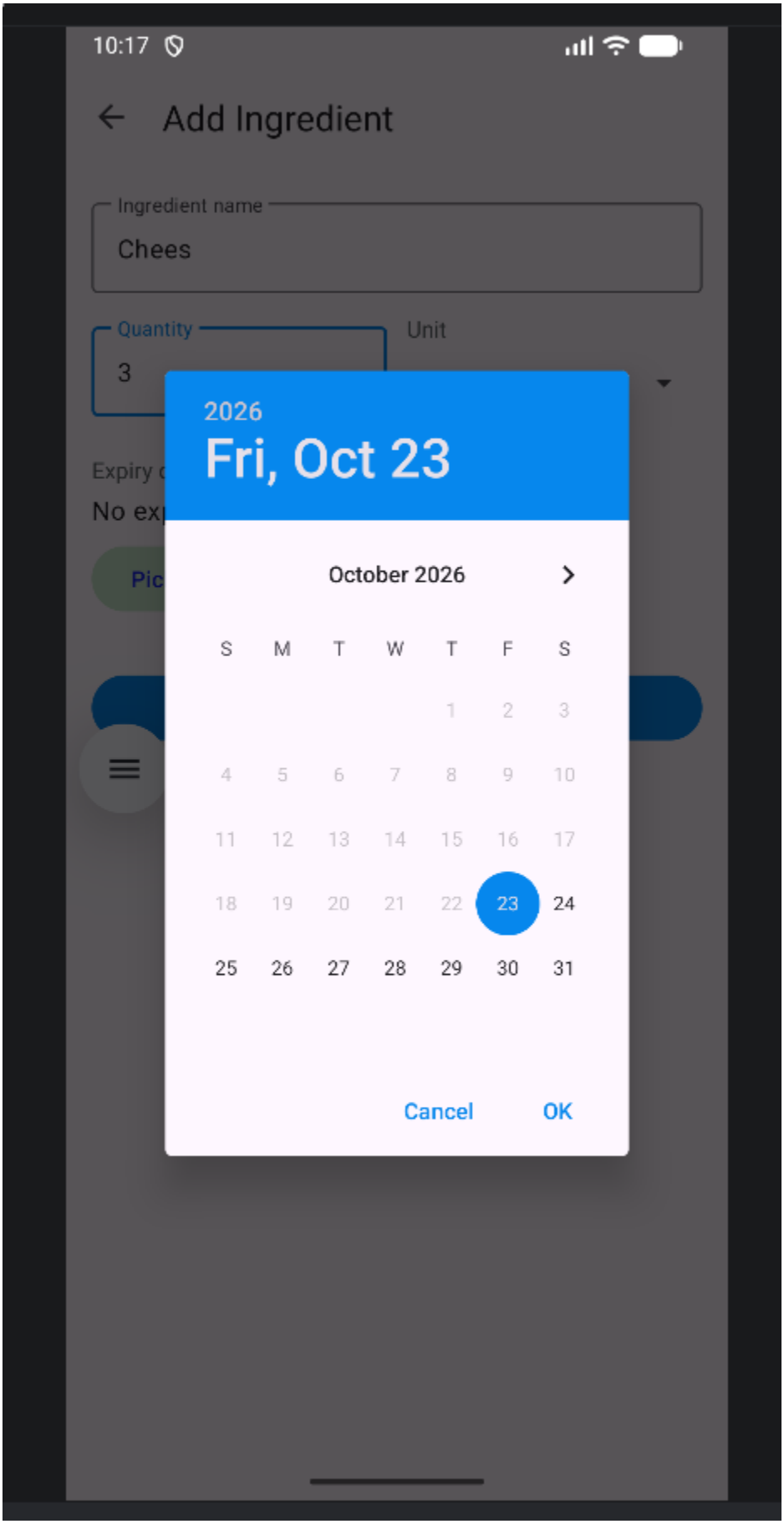

Settings
ALERTS
Expiring-soon alerts
Show a banner on the pantry screen when ingredients are about to expire.
Warn me this many days before expiry
3 days
UNITS
Default unit for new ingredients
g
kg
ml
L
tsp
tbsp
cup
pcs
cook
Pantry
Recipes
Settings

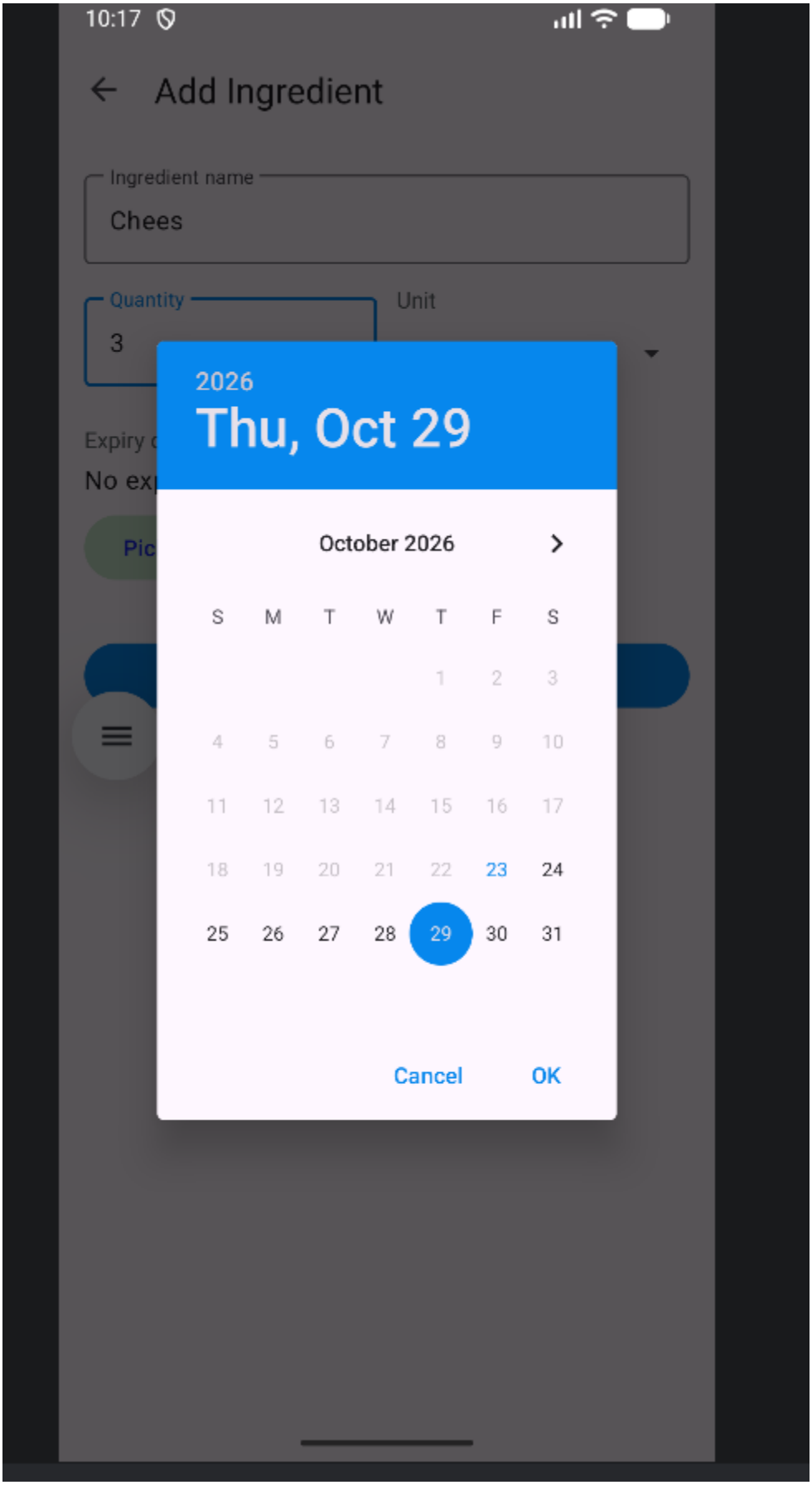

10:16
Settings
ALERTS
Expiring-soon alerts
Show a banner on the pantry screen when ingredients are about to expire.
Warn me this many days before expiry
3 days
UNITS
Clear pantry?
This deletes every ingredient in your pantry. Recipes are not affected.
Cancel
Clear
ABOUT
Smart Pantry Manager suggests only recipes you can cook with what you already have, to help cut food waste.
Pantry
Recipes
Settings

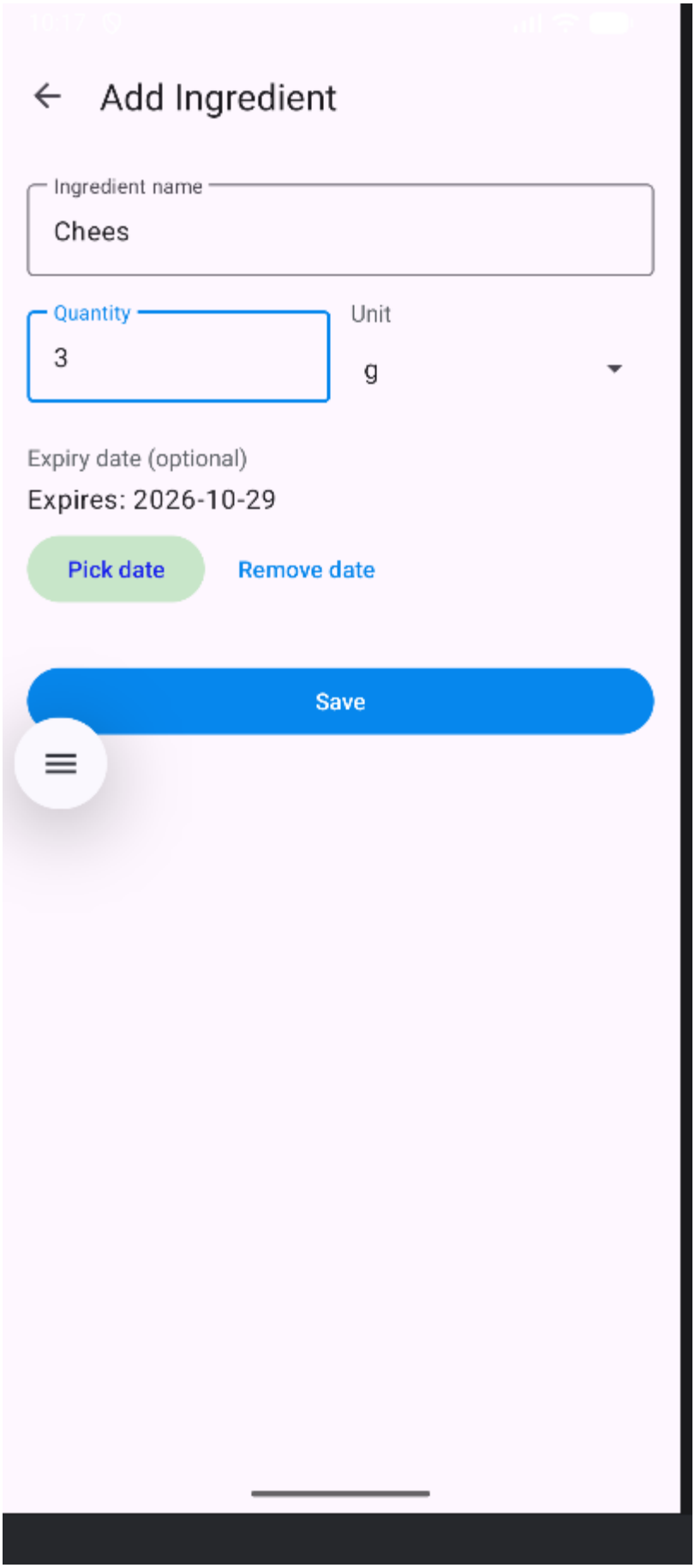

Settings
ALERTS
Expiring-soon alerts
Show a banner on the pantry screen when ingredients are about to expire.
Warn me this many days before expiry
3 days
UNITS
Default unit for new ingredients
g
DATA
Clear pantry
ABOUT
Smart Pantry Manager suggests only recipes you can cook with what you already have, to help cut food waste.
Pantry cleared
Pantry
Settings

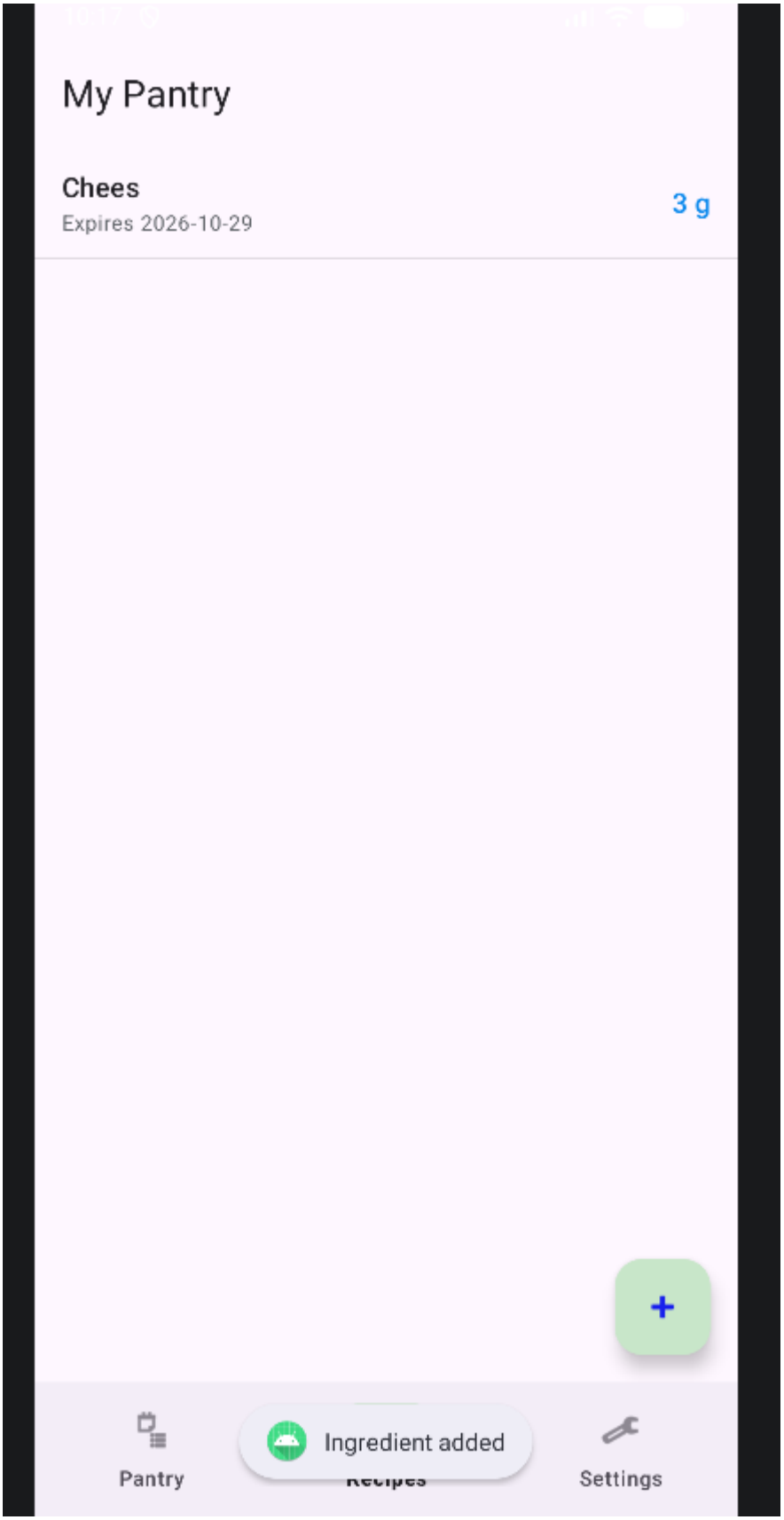

Add Ingredient
Ingredient name
Chees
Quantity
3
Unit
g
Expiry date (optional)
No expiry date
Pick date
Save

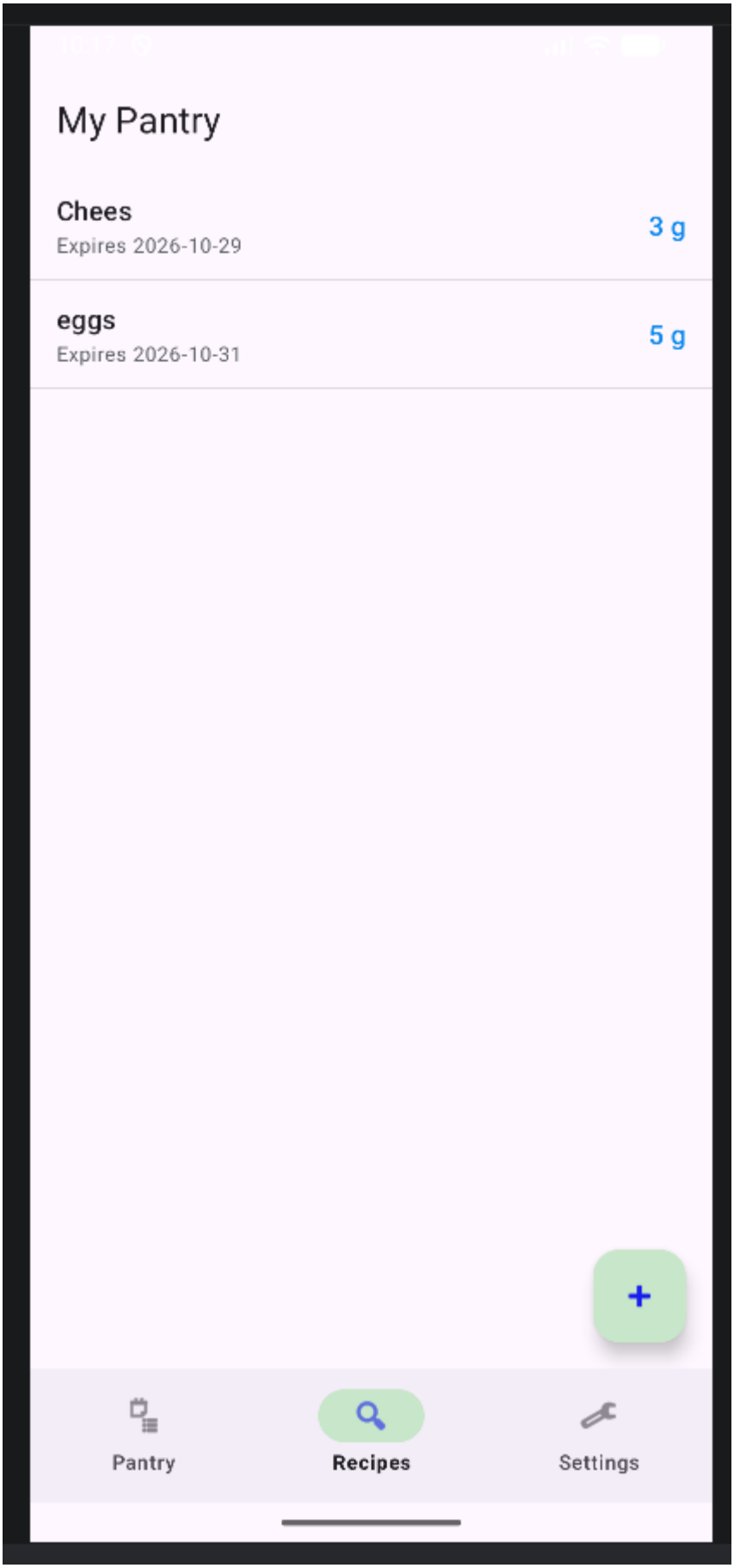

Add Ingredient
Ingredient name
Chees
Quantity
3
Unit
g
kg
ml
L
tsp
tbsp
cup
pcs
Expiry date (optional)
No expiry date
Pick date

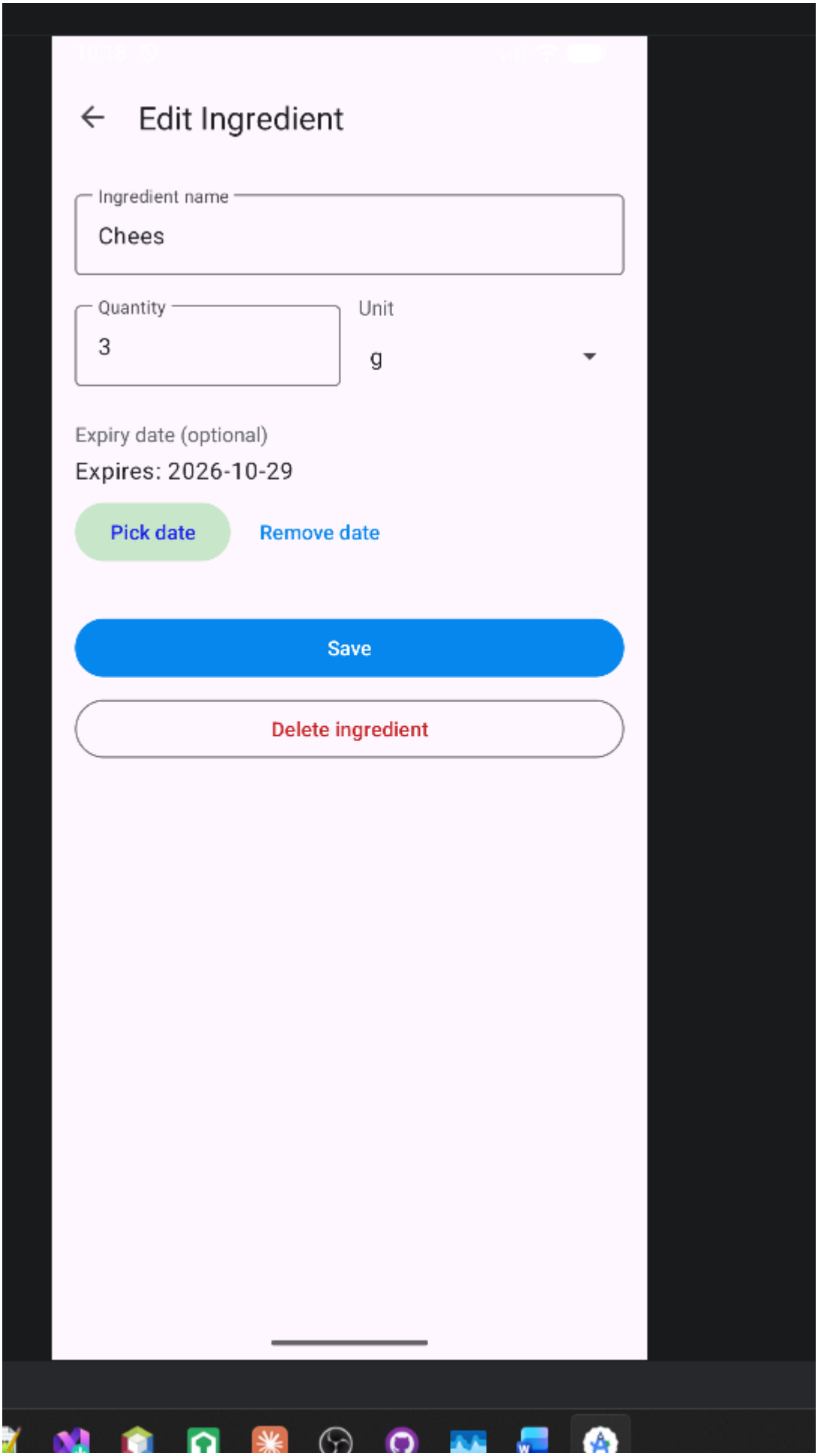

10:17
Add Ingredient
Ingredient name
Chees
Quantity
3
Unit
2026
Fri, Oct 23
October 2026
S M T W T F S
1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
Cancel
OK

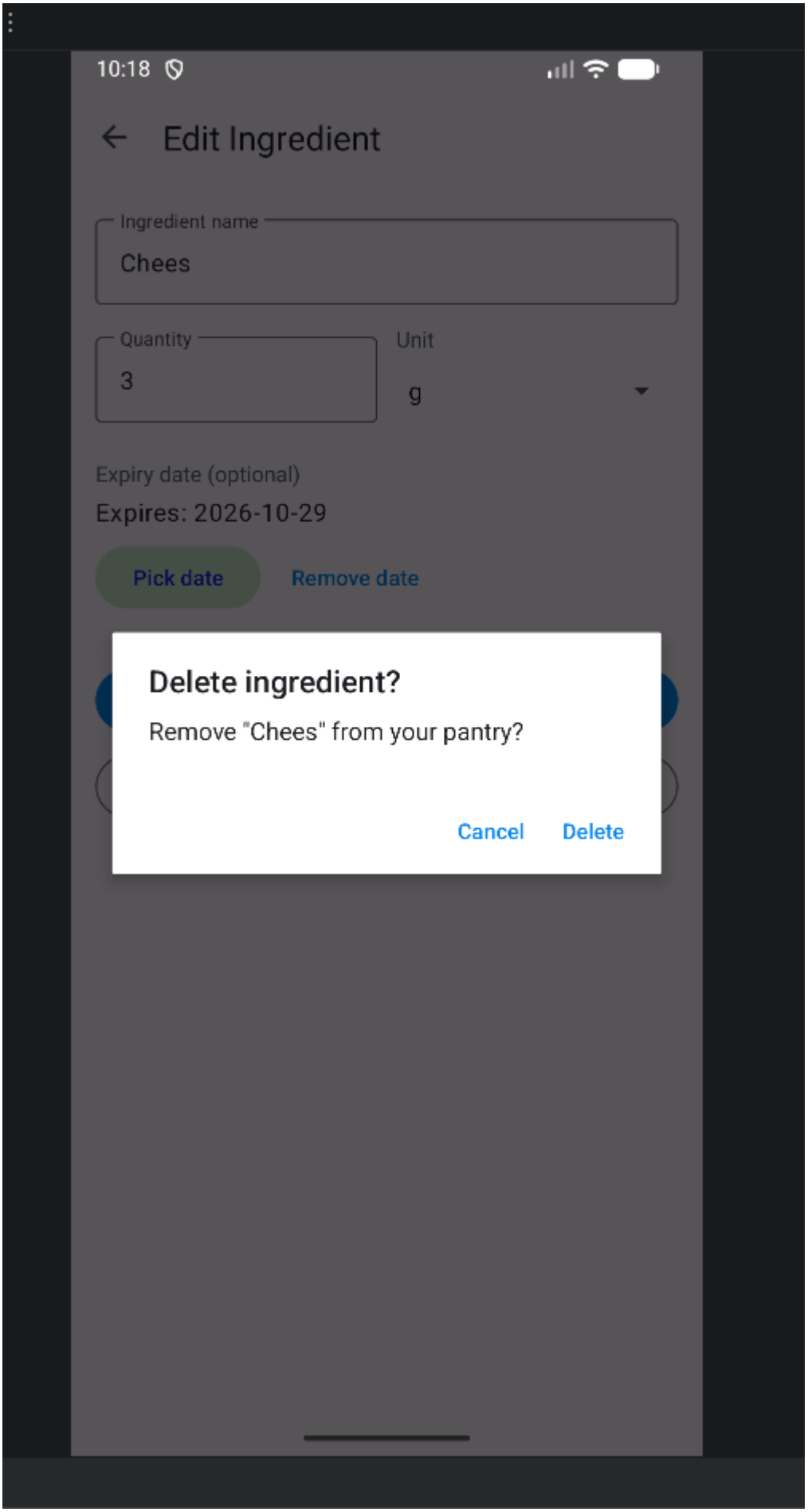

10:17
Add Ingredient
Ingredient name
Chees
Quantity
3
Unit
2026
Thu, Oct 29
October 2026
S M T W T F S
1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
Cancel
OK

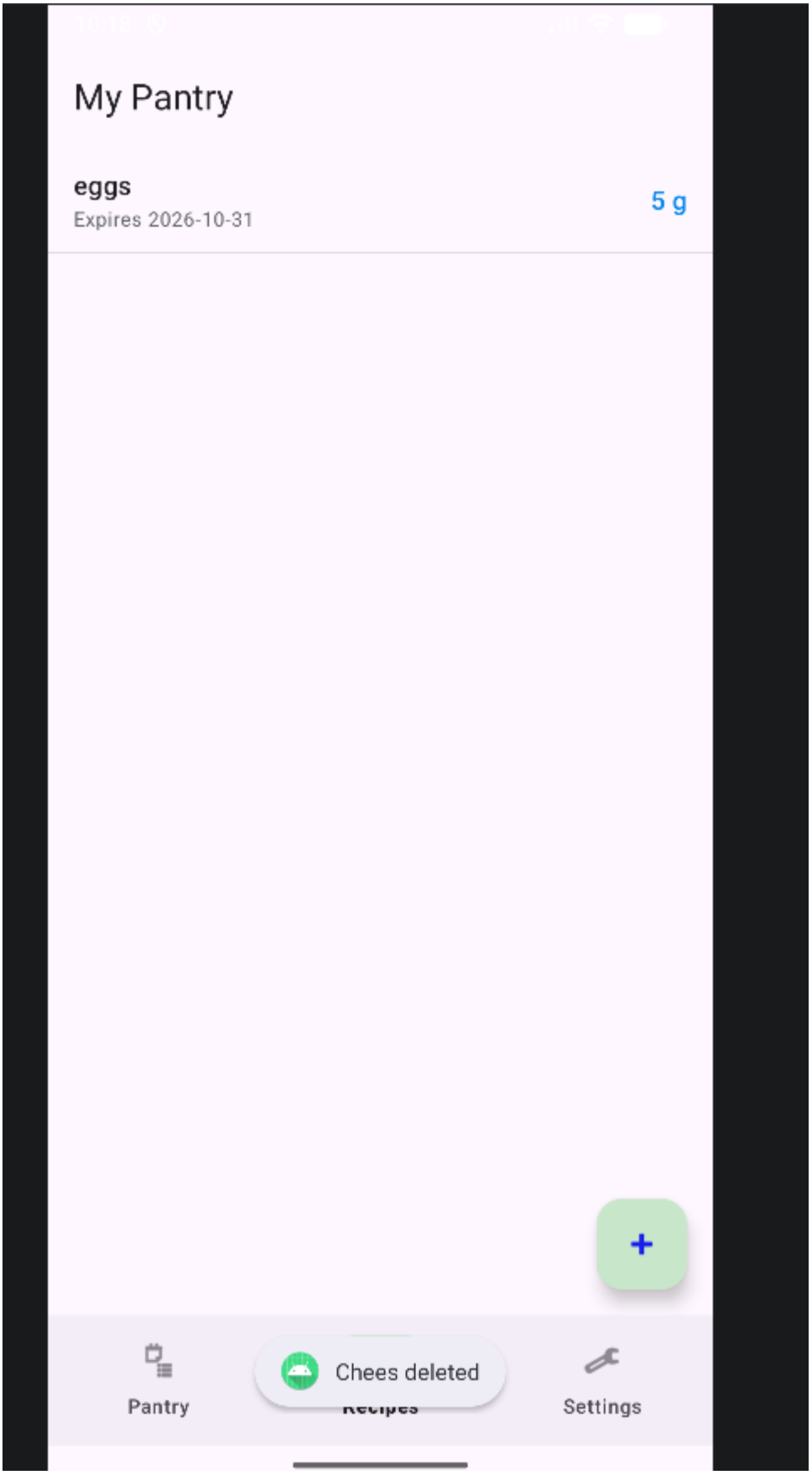

Add Ingredient
Ingredient name
Chees
Quantity
3
Unit
g
Expiry date (optional)
Expires: 2026-10-29
Pick date
Remove date
Save

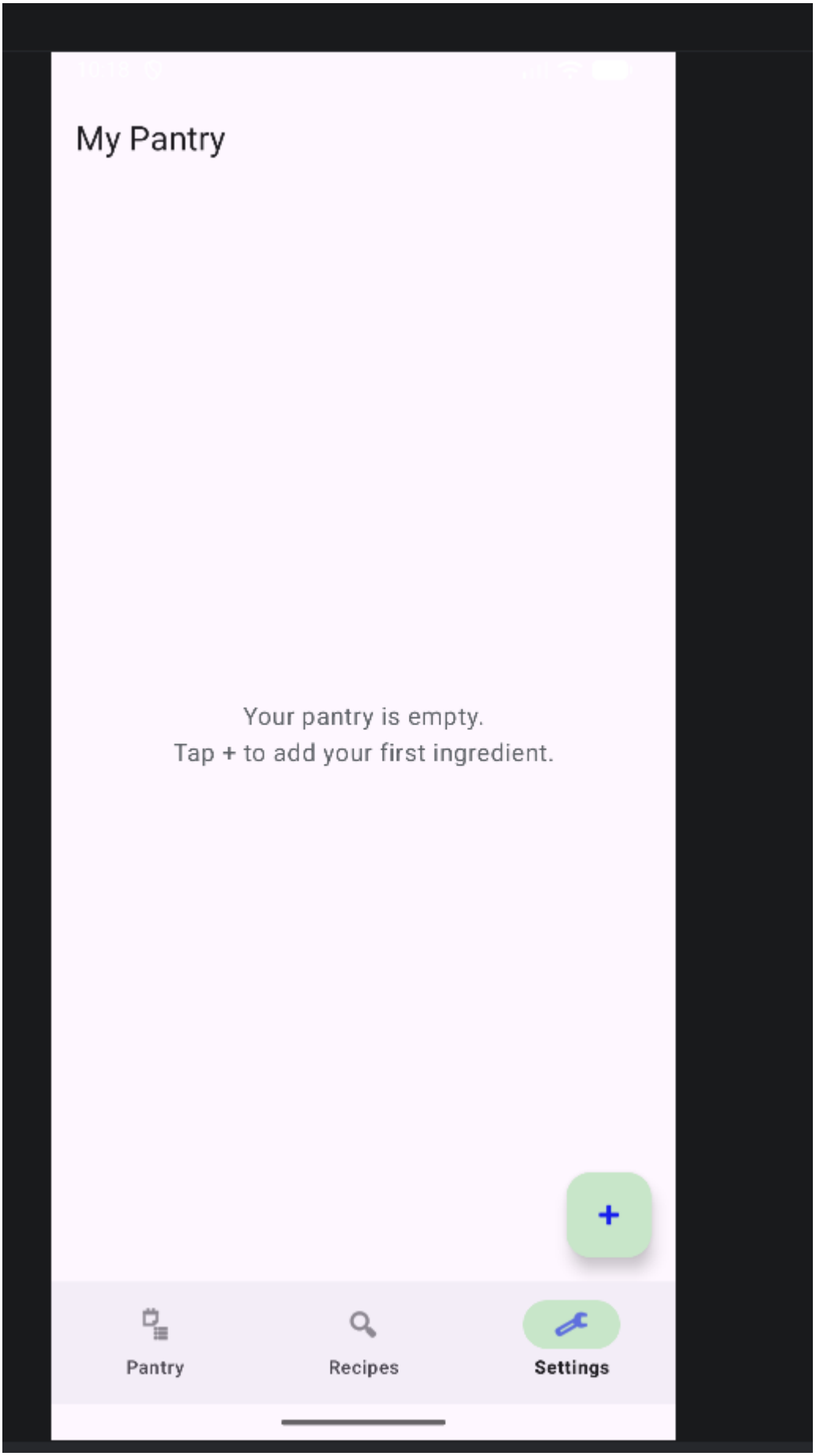

My Pantry
Chees
Expires 2026-10-29
3 g
Ingredient added
Pantry
Settings

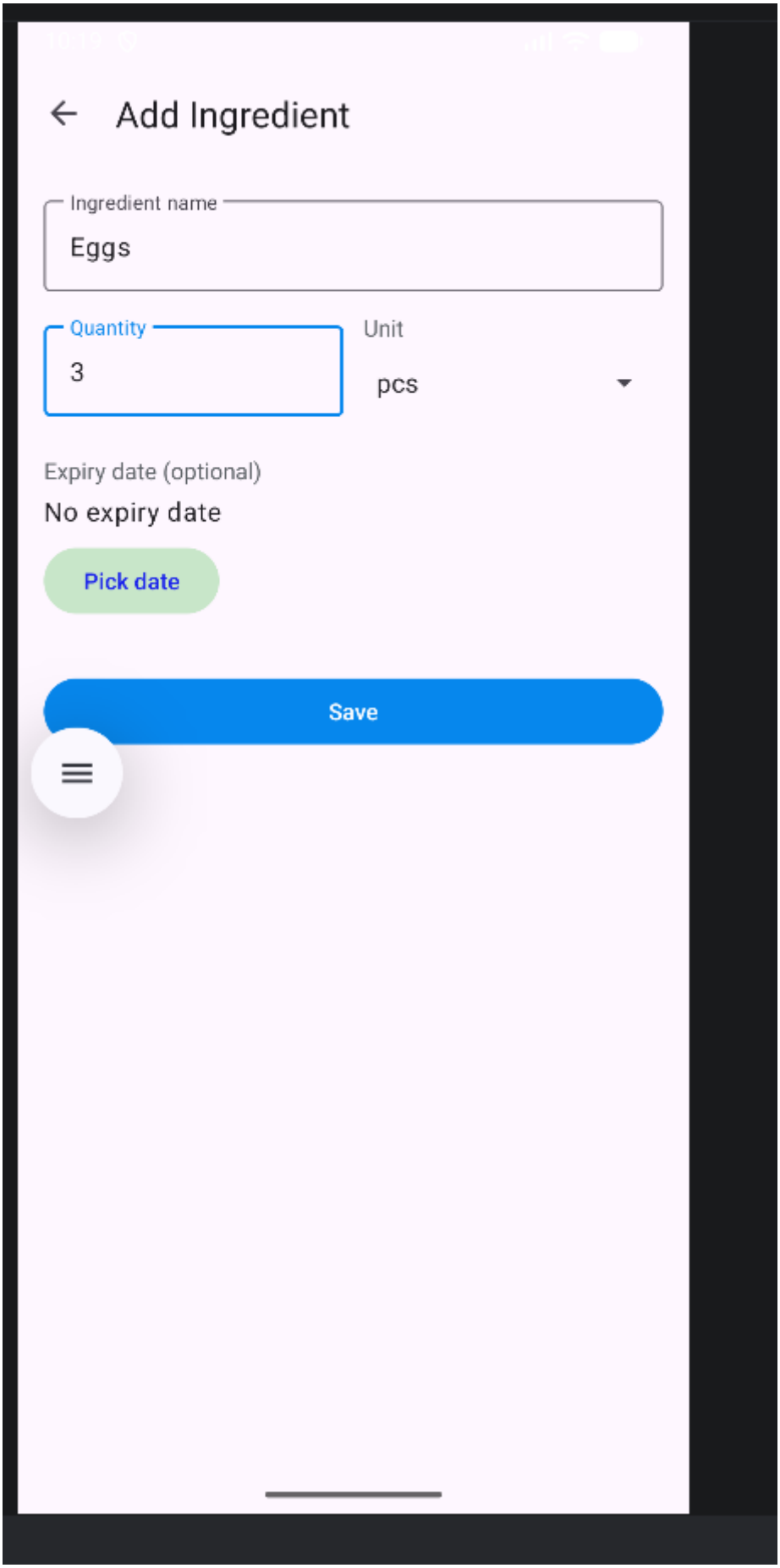

My Pantry
Chees
Expires 2026-10-29
3 g
eggs
Expires 2026-10-31
5 g
+
Pantry
Recipes
Settings

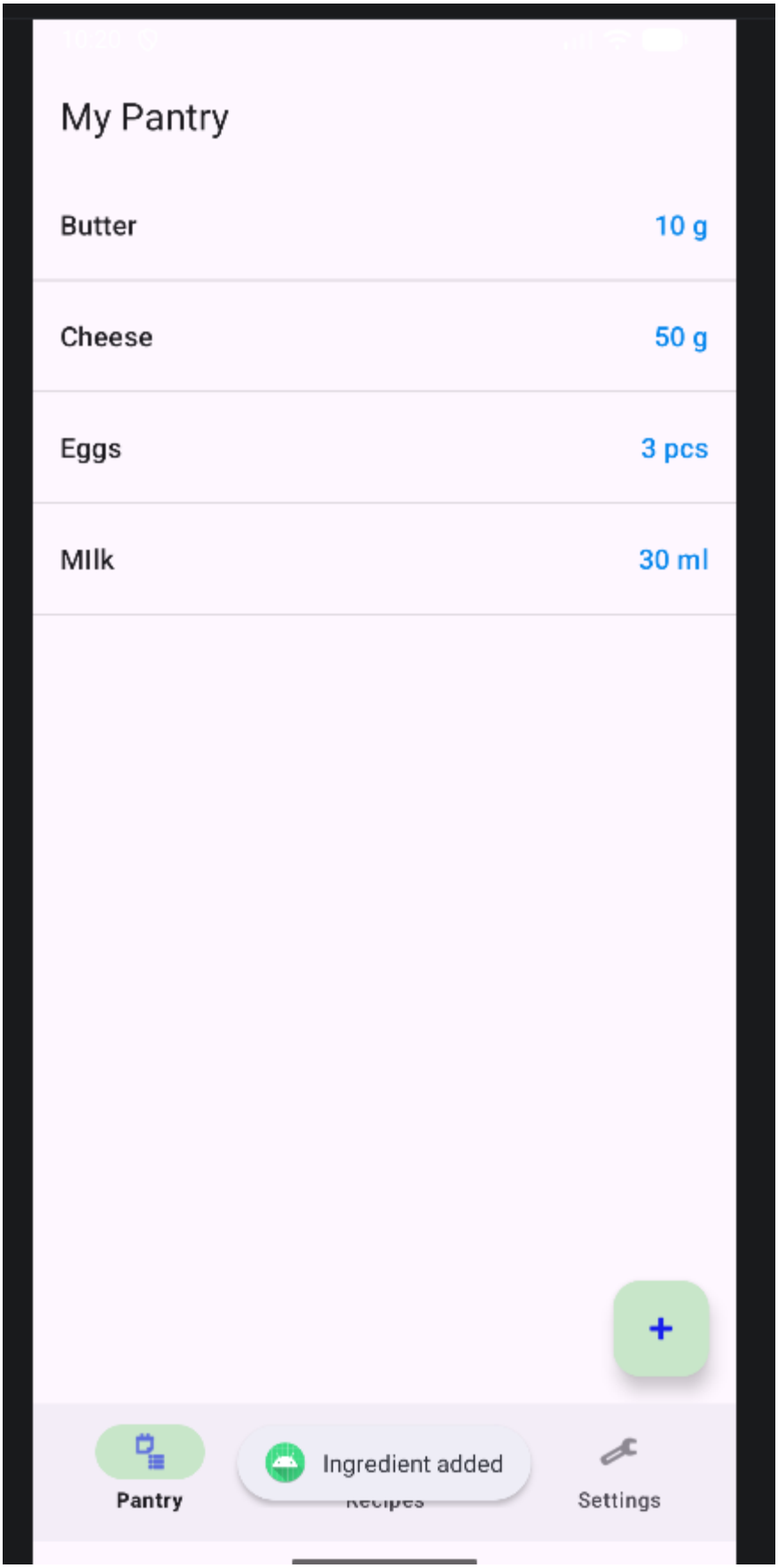

Edit Ingredient
Ingredient name
Chees
Quantity
3
Unit
g
Expiry date (optional)
Expires: 2026-10-29
Pick date
Remove date
Save
Delete ingredient

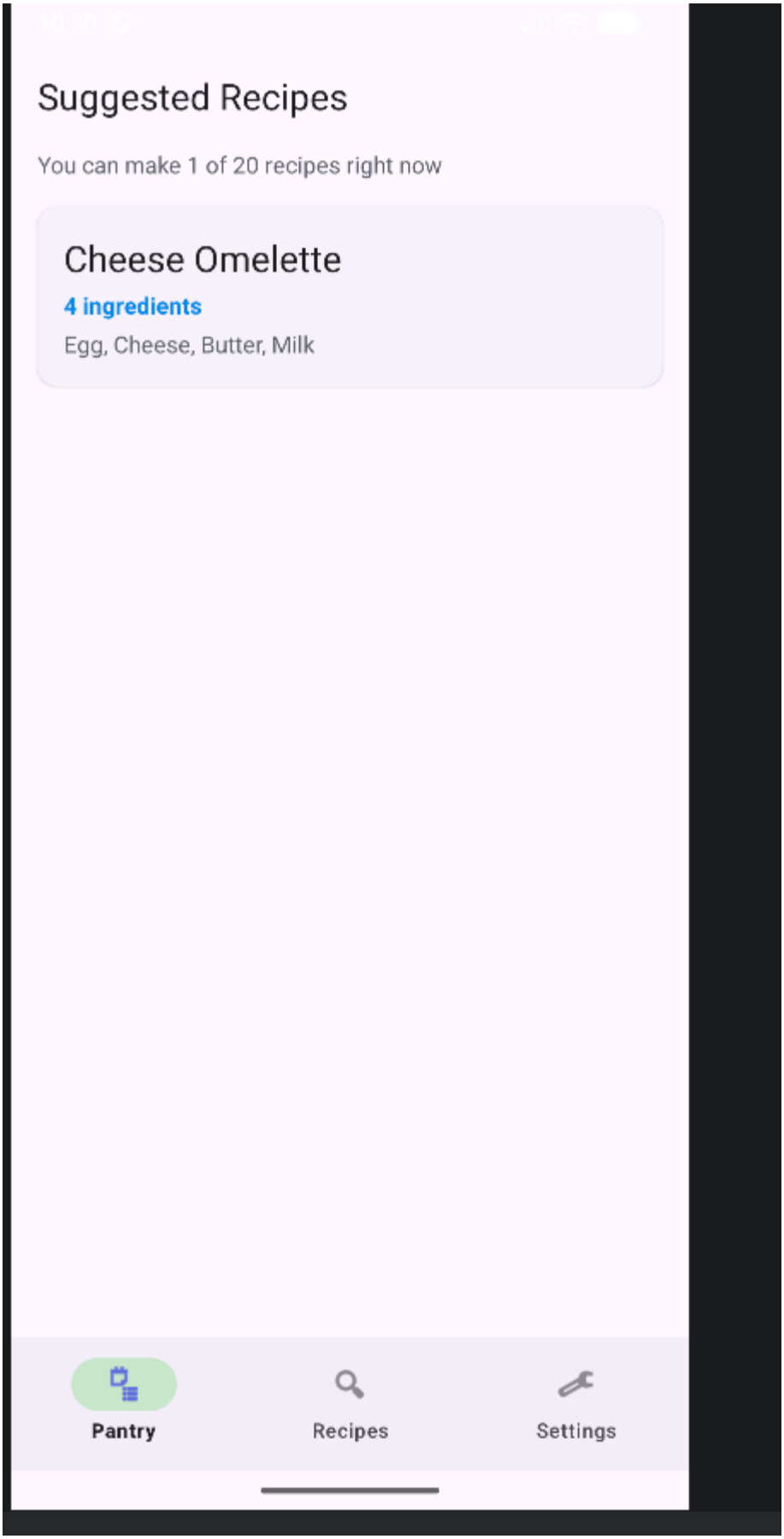

10:18
Edit Ingredient
Ingredient name
Chees
Quantity
3
Unit
g
Expiry date (optional)
Expires: 2026-10-29
Pick date
Remove date
Delete ingredient?
Remove "Chees" from your pantry?
Cancel
Delete

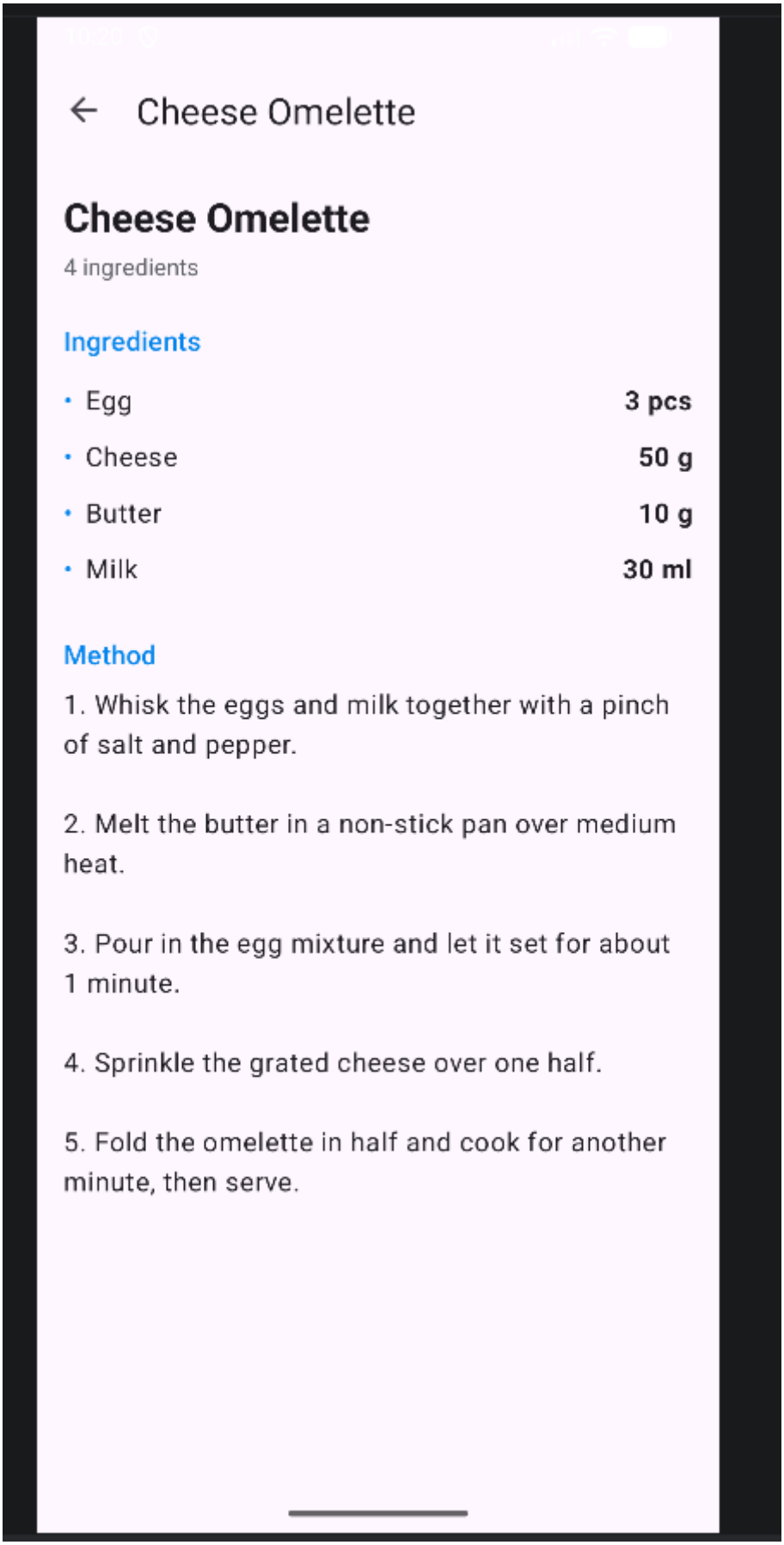

My Pantry
eggs
Expires 2026-10-31
5 g
Chees deleted
Pantry
Settings

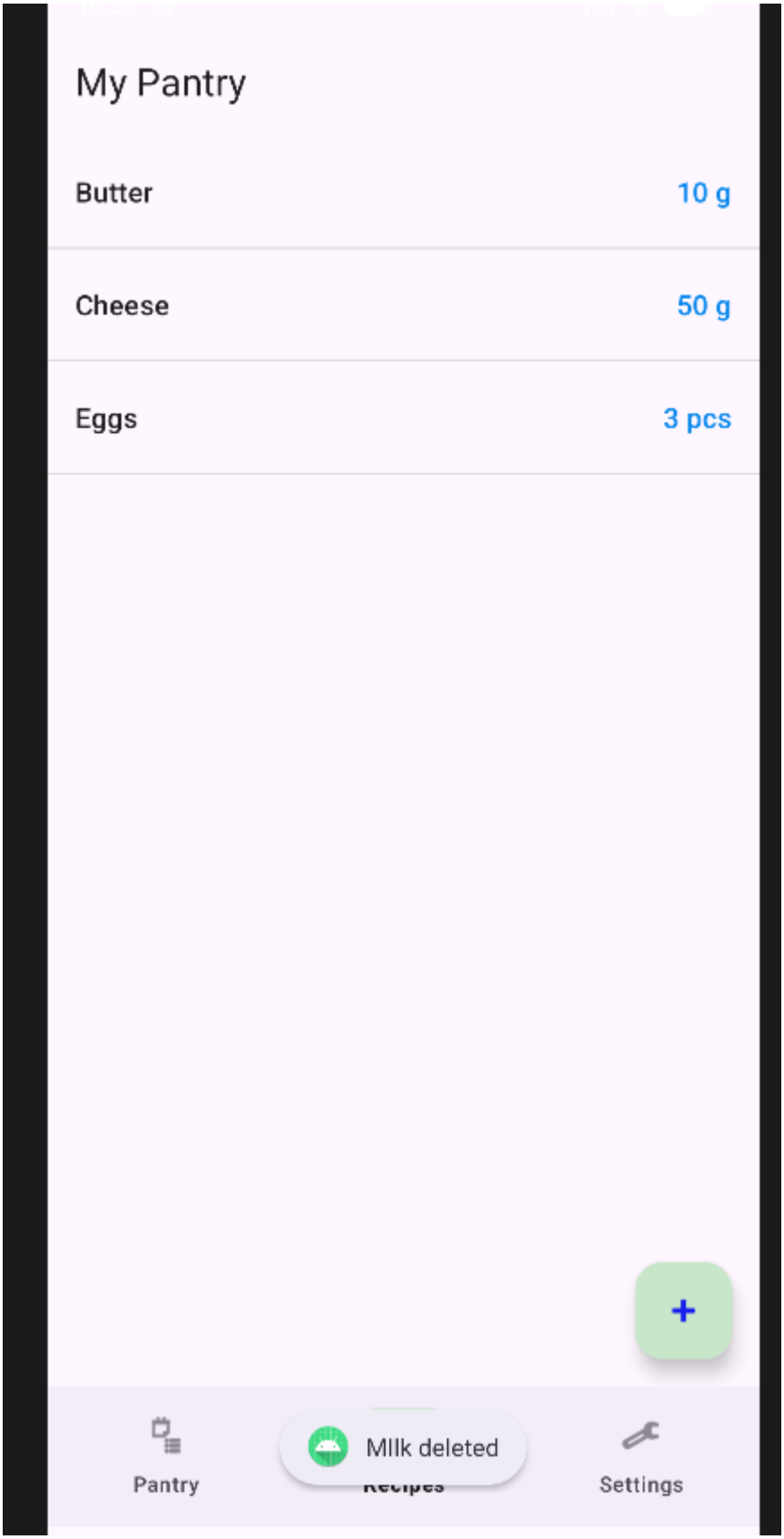

My Pantry
Your pantry is empty.
Tap + to add your first ingredient.
Pantry
Recipes
Settings


Add Ingredient
Ingredient name
Eggs
Quantity
3
Unit
pcs
Expiry date (optional)
No expiry date
Pick date
Save


My Pantry
Butter
10 g
Cheese
50 g
Eggs
3 pcs
Milk
30 ml
Ingredient added
Pantry
Settings


Suggested Recipes
You can make 1 of 20 recipes right now
Cheese Omelette
4 ingredients
Egg, Cheese, Butter, Milk
Pantry
Recipes
Settings


Cheese Omelette
Cheese Omelette
4 ingredients
Ingredients
Egg 3 pcs
Cheese 50 g
Butter 10 g
Milk 30 ml
Method
1. Whisk the eggs and milk together with a pinch of salt and pepper.
2. Melt the butter in a non-stick pan over medium heat.
3. Pour in the egg mixture and let it set for about 1 minute.
4. Sprinkle the grated cheese over one half.
5. Fold the omelette in half and cook for another minute, then serve.


My Pantry
Butter
10 g
Cheese
50 g
Eggs
3 pcs
Milk deleted
Pantry
Settings


My Pantry
1 item(s) expired or expiring within 3 day(s)
Eggs
Expires 2026-10-23
3 pcs
+
Pantry
Recipes
Settings

## 3. KEY CODE SNIPPETS:

Strict-matching rule RecipeMatcher.java:

```
public static boolean canMake(Recipe recipe, Map<String, Double> stock) {
    List<RecipeIngredient> required = recipe.getIngredients();
    if (required.isEmpty()) {
        return false; // a recipe with no ingredient data can't be verified
    }
    for (RecipeIngredient ingredient : required) {
        String key = stockKey(ingredient.getNormalizedName(), ingredient.getUnit());
        Double available = stock.get(key);
        if (available == null) {
            return false; // ingredient not in the pantry at all
        }
        double needed = UnitManager.toBaseUnit(ingredient.getQuantity(), ingredient.getUnit());
        if (available + EPSILON < needed) {
            return false; // in the pantry, but not enough of it
        }
    }
    return true; // every single ingredient is covered
}
```

What it does: checks one recipe against the pantry. The method returns false as soon as any ingredient is missing or there is not enough of it, so partial matches can never be suggested.

Why it was written this way: the pantry is converted into a HashMap beforehand, so each lookup is fast. A small tolerance (EPSILON) prevents floating-point rounding errors from rejecting exact amounts. The class contains no Android code, so it is covered by 11 JUnit tests, including the brief's example of a recipe with 4 of 5 ingredients.

Database creation and seeding DatabaseHelper.java:

```
@Override
public void onConfigure(SQLiteDatabase db) {
    super.onConfigure(db);
    db.setForeignKeyConstraintsEnabled(true); // off by default in SQLite
}

@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL(PantryTable.CREATE_TABLE);
    db.execSQL(RecipeTable.CREATE_TABLE);
    db.execSQL(RecipeIngredientTable.CREATE_TABLE);
    db.execSQL(RecipeIngredientTable.CREATE_INDEX);
    RecipePreLouder.seed(db); // pre-load the 20 starter recipes
}
```

What it does: onCreate() runs only once, the first time the database file is created. It builds the three tables and loads the 20 recipes, which meets the brief's requirement for recipes seeded on first run. onConfigure() runs every time the database opens and switches on foreign keys, so ON DELETE CASCADE works.

Why it was written this way: DatabaseHelper is a singleton, so the whole app shares one connection. All SQL statements are constants in DatabaseSchema, so table and column names are typed in only one place. Android runs onCreate() inside a transaction, so a failure during seeding cannot leave half-filled tables.

CRUD with parameterised queries PantryDao.java:

```
public List<PantryItem> getAll() {
    List<PantryItem> items = new ArrayList<>();
    SQLiteDatabase db = dbHelper.getReadableDatabase();
    try (Cursor cursor = db.query(PantryTable.TABLE_NAME, null, null, null,
            null, null, PantryTable.COL_NAME + " COLLATE NOCASE ASC")) {
```

```
        while (cursor.moveToNext()) {
            items.add(fromCursor(cursor));
        }
    }
    return items;
}

public int delete(long id) {
    SQLiteDatabase db = dbHelper.getWritableDatabase();
    return db.delete(PantryTable.TABLE_NAME, PantryTable._ID + " = ?",
            new String[]{String.valueOf(id)});
}
```

What it does: getAll() reads every pantry item, sorted alphabetically while ignoring capital letters (COLLATE NOCASE). delete() removes one item by its ID. Together with insert(), getById() and update(), these methods provide the full CRUD functionality.

Why it was written this way: Activities never contain SQL; they call these DAO methods instead. Queries use ? placeholders rather than joining user input into the SQL string, which protects against SQL injection. try-with-resources closes each Cursor automatically to prevent memory leaks.

Intents and the Activity lifecycle PantryListActivity.java:

```
@Override
protected void onResume() {
    super.onResume();
    loadPantry(); // runs every time the screen returns to the foreground
}

@Override
```

```
public void onItemClick(PantryItem item) {
    Intent intent = new Intent(this, AddEditIngredientActivity.class);
    intent.putExtra(AddEditIngredientActivity.EXTRA_ITEM_ID, item.getId());
    startActivity(intent);
}
// AddEditIngredientActivity.onCreate(): read the extra to choose the mode
itemId = getIntent().getLongExtra(EXTRA_ITEM_ID, NO_ID);
if (isEditMode()) {
    setTitle(R.string.title_edit_ingredient);
    btnDelete.setVisibility(View.VISIBLE);
}
```

What it does: tapping a pantry row starts the edit screen with an explicit Intent and passes the item's database ID as an extra. The edit screen reads the extra to decide between add mode (no ID) and edit mode (ID present), so one form handles both Create and Update.

Why it was written this way: passing only the ID keeps the Intent small, and the edit screen loads fresh data from the database. Reloading the list in onResume() instead of onCreate() means the list updates automatically after the user saves or deletes and returns to the screen.

Input validation AddEditIngredientActivity.java:

```
if (quantityText.isEmpty()) {
    tilQuantity.setError(getString(R.string.error_quantity_required));
    isValid = false;
} else {
    try {
        // South African keyboards often use a comma as the decimal separator
        quantity = Double.parseDouble(quantityText.replace(',', '.'));
        if (quantity <= 0) {
            tilQuantity.setError(getString(R.string.error_quantity_positive));
```

```
            isValid = false;
        }
    } catch (NumberFormatException e) {
        tilQuantity.setError(getString(R.string.error_quantity_invalid));
        isValid = false;
    }
}
if (!isValid) {
    return; // stay on the form so the user can fix the errors
}
```

What it does: validates the quantity before anything is saved. An empty field, a value of 0 or less, or text that is not a number shows an error message directly under the field, and nothing is written to the database. The name field is validated in the same way (required, letters only, maximum 40 characters).

Why it was written this way: catching NumberFormatException prevents a crash on invalid input. Replacing a comma with a full stop supports South African keyboards, where "1,5" is a normal way to type one and a half. The CHECK (quantity > 0) constraint in the database is a second layer of protection.

## 4. CHALLENGES AND THE SOLUTION:

Template code blocked the build and unit tests:

Problem: when I first ran the RecipeMatcherTest unit tests, the build failed with "cannot find symbol variable activity_main" in MainActivity.java. The tests themselves were correct. The problem was the template MainActivity created by Android Studio, which still referred to the template layout activity_main.xml that I had replaced. Gradle compiles the whole app module before running any tests, so one broken file blocked everything.

Solution: I removed the template MainActivity and registered PantryListActivity as the launcher Activity in AndroidManifest.xml. After this, the project compiled and all 11 tests passed. I learned that every compile error in a module must be fixed before anything in it can run, even code that is unrelated to the error.

App crashed on launch because of an SQL syntax error:

Problem: after adding the screens, the app closed immediately every time it opened. Logcat showed a FATAL EXCEPTION caused by SQLiteException: near "CASE": syntax error in the query ORDER BY name COLLATE NO CASE ASC, called from PantryDao.getAll() in onResume().

Solution: the collation name had been typed with a space: NO CASE instead of NOCASE. SQLite therefore read NO as the collation and could not understand CASE. I corrected it to COLLATE NOCASE, then used Find in Files to check for the same mistake elsewhere in the project. The main lesson was that SQL written inside Java strings is not checked by the compiler, so errors only appear at runtime. Reading the Caused by: line and the first line from my own package in Logcat led me straight to the faulty line.

Making strict matching robust to real-world input:

Problem: a naive exact-string comparison would fail on trivial differences. For example, "Tomatoes" in the pantry would not match "tomato" in a recipe, 1 kg of flour would not satisfy a recipe needing 500 g, and the same food has different South African names, such as "mielie meal" and "maize meal". The brief states that naive matching will be marked down.

Solution: I split the problem into two small classes:

- IngredientManager normalises names (lowercase, trim, singular form and a synonym list).
- UnitManager converts quantities to base units within a category (grams, millilitres or pieces) and refuses to compare different categories.

The normalised name is stored in the database when a row is saved. To make comparisons reliable, I seeded the recipes in grams for dry food, millilitres for liquids and pieces for countable items. I wrote JUnit tests for each case (plurals, capitals, unit conversion, mismatched categories, synonyms and duplicate pantry rows) to prove that the rule works before connecting it to the user interface.

# 5. CONCLUSION:

Conclusion:

Smart Pantry Manager meets the requirements of the assignment brief:

- It is a Java Android application with five screens connected by Intents and a bottom navigation bar.
- It provides full CRUD on pantry items, stored in SQLite and persisting between sessions.
- It displays dynamic lists with RecyclerViews and custom Adapters, and validates user input.
- Its core strict-matching rule suggests only recipes the user can genuinely cook with what they already have.

It handles common real-world differences in ingredient names and units, and it gives clear feedback when no recipes match.

What I learned:

- How the Activity lifecycle affects data: reloading lists in onResume() keeps every screen up to date.
- How to design and build an SQLite database with SQLiteOpenHelper, including foreign keys, constraints and first-run seeding.
- How RecyclerView, Adapters and ViewHolders work together to display database data efficiently.
- How to use explicit Intents with extras to pass data between screens.
- Why keeping business logic separate from Android code makes it testable, and how to write JUnit tests.
- How to debug runtime crashes using Logcat stack traces, and how to work with Git branches, commits and merges.

What I would improve with more time:

- "Almost There" list: a separate list of recipes missing only one ingredient, clearly separated from the strict suggestions.
- "Cook this" button: subtracts the used ingredients from the pantry after cooking.
- Expiry notifications: reminders via WorkManager, even when the app is closed.
- Background database access: move database work to a background thread, or migrate to the Room library, for larger datasets.
- Custom recipes: let users add their own recipes, and support a dark theme.

Bibliography

Android Developers (n.d.a) Create dynamic lists with RecyclerView. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 23 October 2026).

Android Developers (n.d.b) Intents and intent filters. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 23 October 2026).

Android Developers (n.d.c) Save data using SQLite. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 23 October 2026).

Android Developers (n.d.d) Save simple data with SharedPreferences. Available at: https://developer.android.com/training/data-storage/shared-preferences (Accessed: 23 October 2026).

Android Developers (n.d.e) The activity lifecycle. Available at: https://developer.android.com/guide/components/activities/activity-lifecycle (Accessed: 23 October 2026).

Android Developers (n.d.f) View logs with Logcat. Available at: https://developer.android.com/studio/debug/logcat (Accessed: 23 October 2026).

Anthropic (2026) Claude [AI assistant]. Available at: https://claude.ai (Accessed: 23 October 2026).

CSIR (2021) 45% of available food supply in South Africa wasted, shows new CSIR study. Council for Scientific and Industrial Research. Available at: https://www.csir.co.za/food-supply-south-africa-wasted-shows-new-csir-study (Accessed: 23 October 2026).

Google (n.d.) Material Design 3. Available at: https://m3.material.io/ (Accessed: 23 October 2026).

JUnit (n.d.) JUnit 4. Available at: https://junit.org/junit4/ (Accessed: 23 October 2026).

SQLite (n.d.a) Datatypes in SQLite: collating sequences. Available at: https://www.sqlite.org/datatype3.html (Accessed: 23 October 2026).

SQLite (n.d.b) SQLite foreign key support. Available at: https://www.sqlite.org/foreignkeys.html (Accessed: 23 October 2026).

United Nations Environment Programme (2024) Food Waste Index Report 2024. Nairobi: UNEP. Available at: https://www.unep.org/resources/publication/food-waste-index-report-2024 (Accessed: 23 October 2026).

Richfield Graduate Institute of Technology (2026) Mobile App Development 700: Practical assignment — Smart Pantry Manager. Unpublished assignment brief.